Pivoting

"The Art of finding a new computer to pwn after pwning the first one."

1	Motivation & Terminology
2	SSH & Port Forwarding in General
3	SSH Local Port Forward
4	SSH Dynamic Port Forward
5	SSH Remote Port Forward
6	tun2socks
7	SSH Pivoting with sshuttle
8	ligolo-ng
9	Conclusion

Tip: click any entry to jump to that section

Pivoting

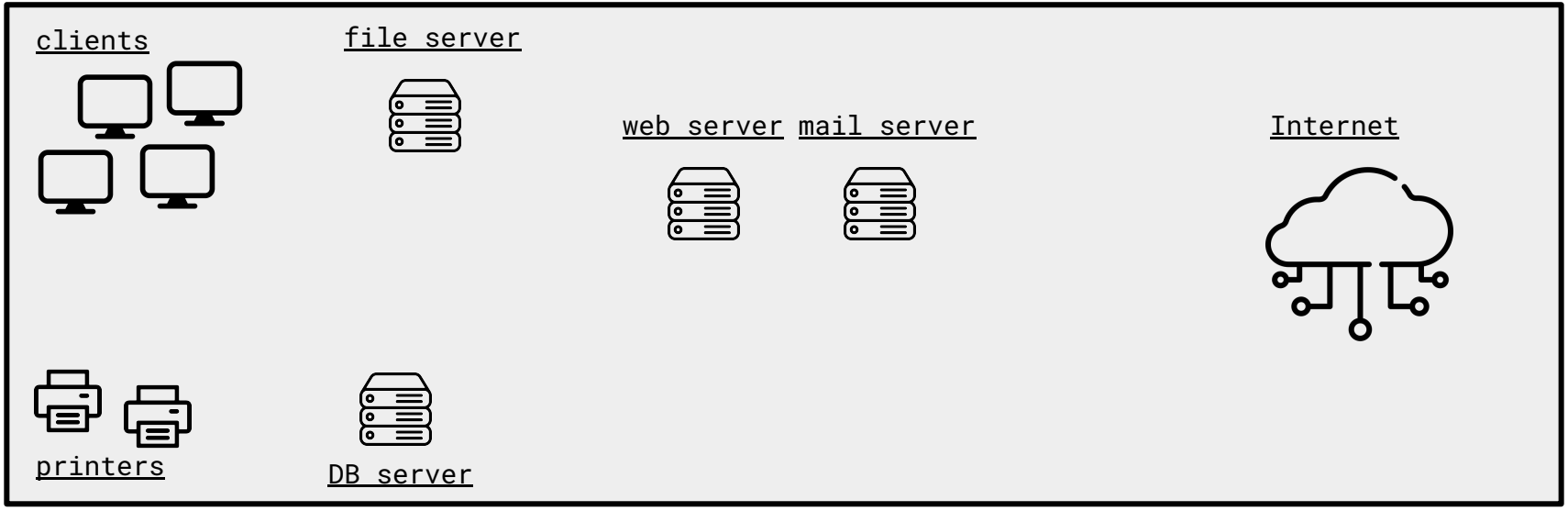
"The Art of finding a new computer to pwn after pwning the first one."

1	Motivation & Terminology
2	SSH & Port Forwarding in General
3	SSH Local Port Forward
4	SSH Dynamic Port Forward
5	SSH Remote Port Forward
6	tun2socks
7	SSH Pivoting with sshuttle
8	ligolo-ng
9	Conclusion

Tip: click any entry to jump to that section

1 motivation & terminology

- these are some of the typical network capable devices you might encounter in an organization
- how would you connect them, given switches and routers?



1 motivation & terminology

- we will definitely need a router as a gateway and border to the Internet, that is clear, but what about the connections between our devices?

switch:  router: 

clients



file server



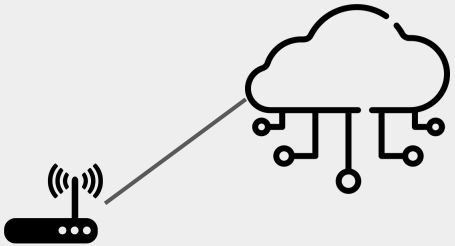
web server



mail server



Internet





printers

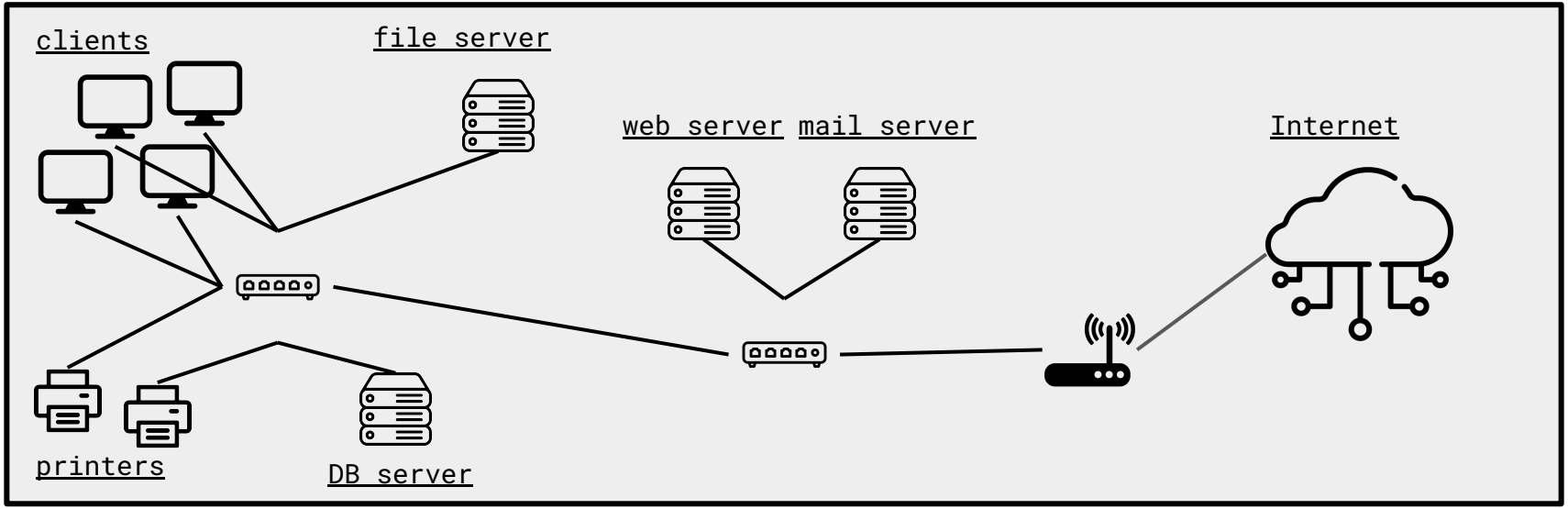


DB server

1 motivation & terminology

- option #1: connect everything with switches → "*flat hierarchy*"
- while this is a possibility, it comes with an array of security disadvantages

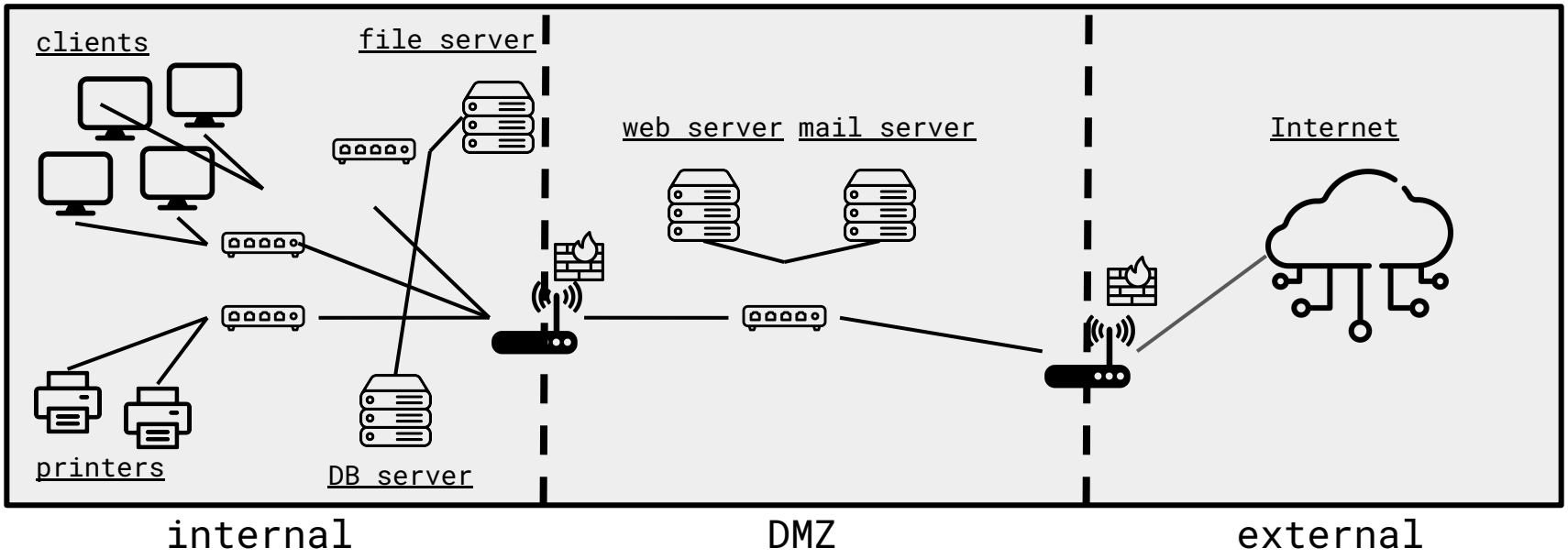
switch:  router: 



- <u>disadvantages of a flat hierarchy</u>
  - monitoring communication between different systems is difficult
  - we cannot differentiate different “security zones”, i.e., differentiating between devices that communicate with external systems directly and those who do not
  - if a system gets pwned, the adversary has direct access to all other systems in the network

- option #2: group systems with similar needs/properties in same networks, while connecting the networks via routers to introduce hierarchy → "*segmentation*"

switch: router:



- <u>security guarantees through segmentation</u>
  - harder to move through the network
  - harder to move through the network undetected
  - allows for proper monitoring and logging of traffic flow
    - into the network
    - out of the network
    - between the different subnetworks
- segmentation is a must

- question: what can I do as a potential attacker to circumvent segmentation or deal with it?
- answer: we can use
  - port forwarding
  - tunneling
  - pivoting

- port forwarding
  - in the same rough category as techniques such as NAT (network address translation) or PAT (port address translation)
  - “any packet that gets sent to this socket (= IP + port) will be forwarded to a different socket”
  - below, you can find a command that would implement such a basic port forward (intuitive example)

```
[user@kali ~]$ nc -l 5555 | nc remote_host 80
```

- tunneling
  - technique in which a PDU (protocol data unit) is encapsulated inside of another PDU; called sending a PDU over a *tunnel*
  - advantages
    - allows data to be transmitted over a network that may not natively support the original protocol (e.g. 4in6, 6in4)
    - typically, tunnel is encrypted and protects the data as it moves over an insecure network
  - below, you can find a command that would implement such a tunnel

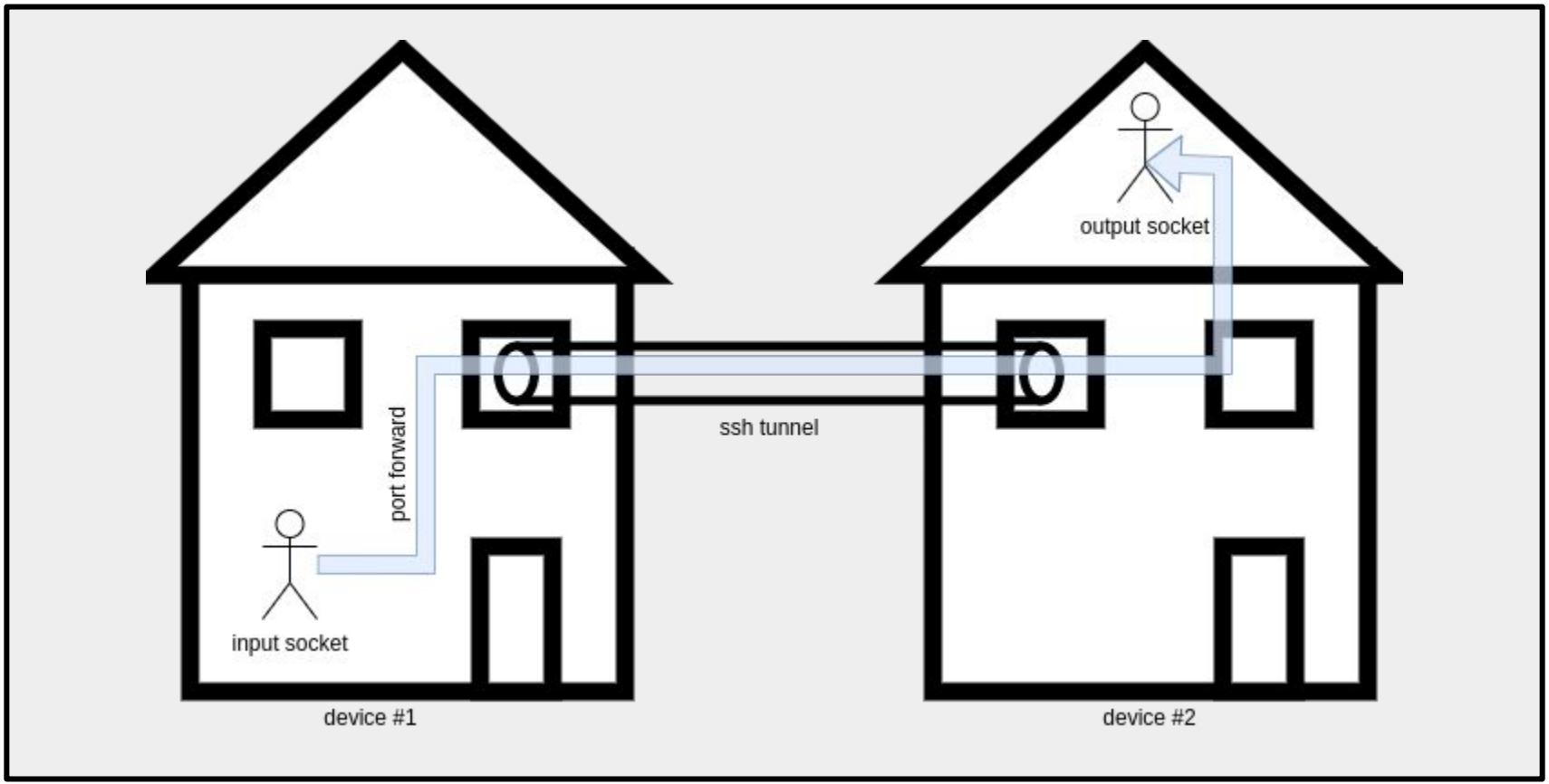
```
[user@kali ~]$ ssh user@device
```

- port forwarding & tunneling in SSH
  - empirically, some individuals will use these terms interchangeably in the context of SSH, which is incorrect (in any context)
  - SSH allows to forward ports over a tunnel, i.e., we can use the tunnel that SSH builds to forward traffic from a socket on one side to the socket on the different side of the tunnel

- <u>port forwarding & tunneling in SSH (intuition)</u>
  - two residencies, each containing multiple residents
  - thick, hollow and opaque PVC pipe between the two houses, from one window into the other
    - from outside, no one can see into the pipe
  - through the pipe, a hose is threaded, which allows one resident to pour water into, arriving at the resident of the other house

- port forwarding & tunneling in SSH (intuition)
  - two residencies, each containing multiple residents → network capable device
  - thick, hollow and opaque PVC pipe between the two houses, from one window into the other
    - from outside, no one can see into the pipe (encrypted) tunnel
  - through the pipe, a hose is threaded, which allows one resident to pour water into, arriving at the resident of the other house port forward
- concepts:
  - tunnel connects two network capable devices
  - port forward connects to ports/sockets
    - port forward between two devices is made possible through a tunnel
    - multiple port forwards can use the same tunnel

1 motivation & terminology



- lateral movement
  - techniques attackers use to move from one compromised system to other systems within the same network
  - example: using credentials found on system A to log into system B
  - goal is to broaden their foothold, escalate privileges, and eventually reach high-value targets or sensitive data

- pivoting
  - a technique in which an attacker will use an already compromised device as a "*pivot point*" to access information, devices and networks that would otherwise be unreachable
  - an example for lateral movement

Pivoting

"The Art of finding a new computer to pwn after pwning the first one."

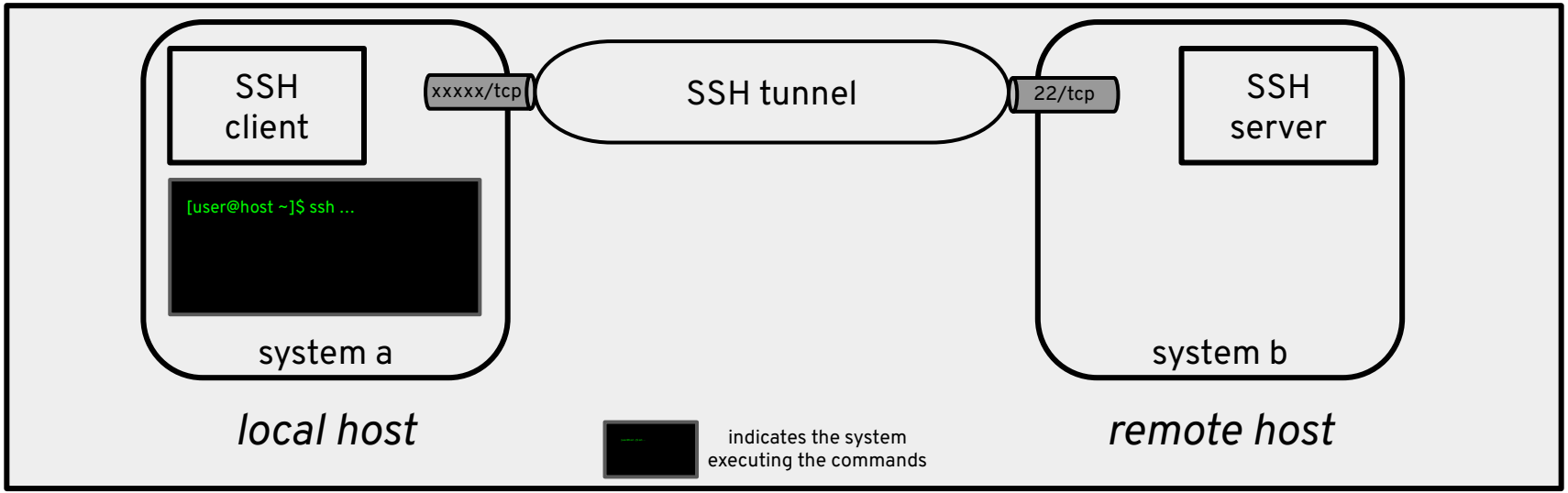
1	Motivation & Terminology
2	SSH & Port Forwarding in General
3	SSH Local Port Forward
4	SSH Dynamic Port Forward
5	SSH Remote Port Forward
6	tun2socks
7	SSH Pivoting with sshuttle
8	ligolo-ng
9	Conclusion

Tip: click any entry to jump to that section

2 SSH & port forwarding in general - why use ssh

- `ssh` is a tool for accessing a remote system with a shell via the SSH ("*Secure Shell*") protocol
  - we refer to the OpenSSH implementation of the SSH utility
- also provides other features, such as port forwarding capabilities
- alternatives exist (`socat`, `rinetd`, ...), but ssh is the standard port forwarding tool for the following reasons:
  - **security**: authentication (password / public-key)
  - **simplicity**: easy to set up & widely available
  - **covertness**: from perspective of monitor software, SSH is quite common & encryption hides malicious commands

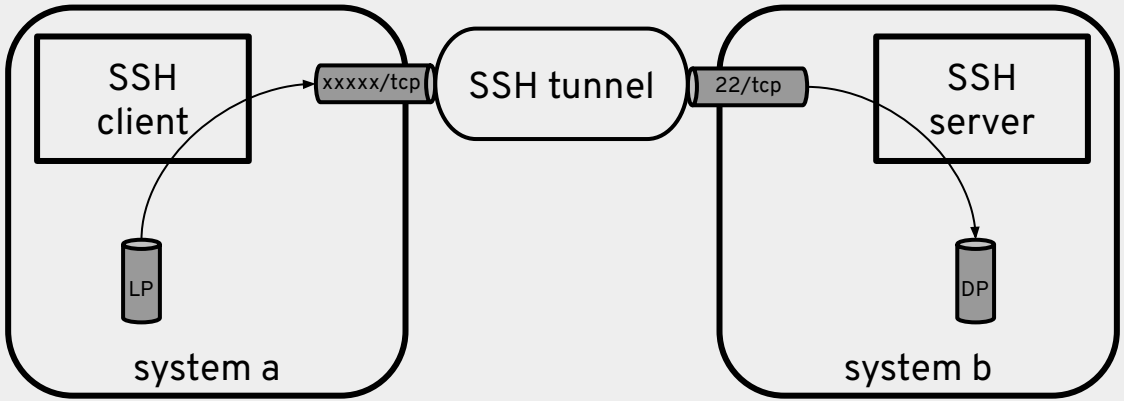
2 SSH & port forwarding in general - *local* vs. *remote*



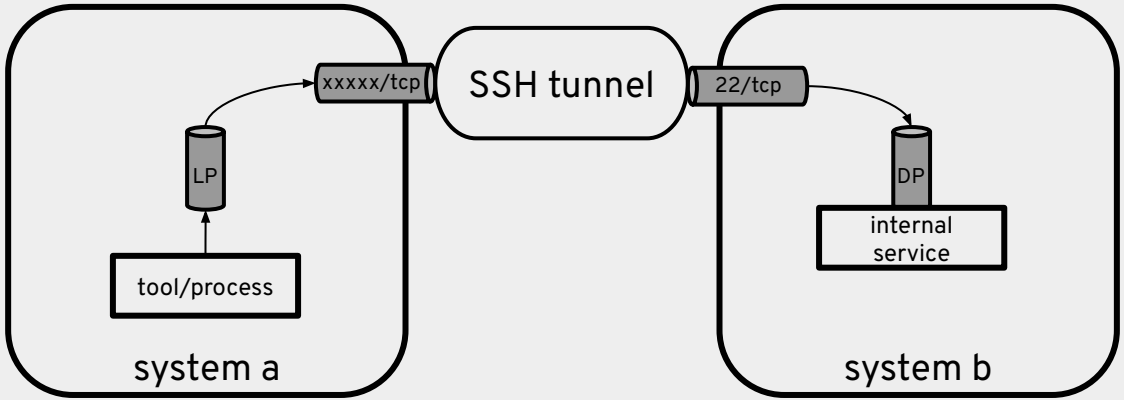
- generally, “local” system is always our attacker system, but we will use a different notion for the context of ssh port forwarding
  - system that is initiating the connection → local
    - “*we need to be on it to issue the* ssh *command* → *“local”*
  - system that is receiving our connection → remote

2 SSH & port forwarding in general - intro

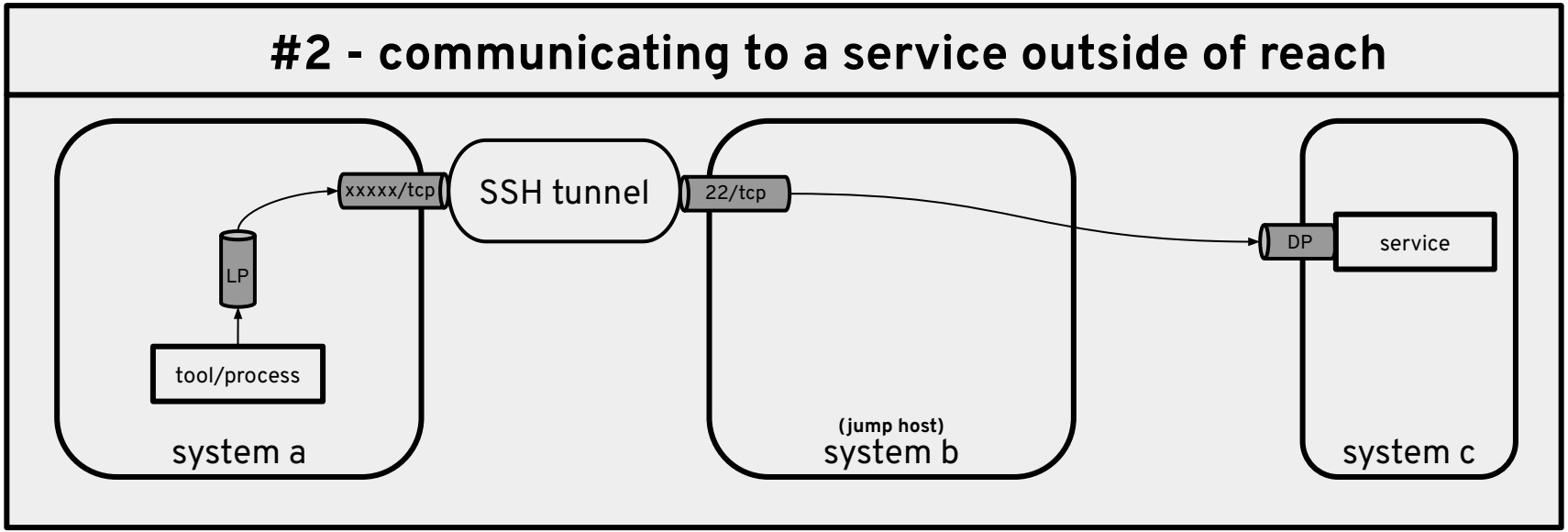
- *port forwarding* techniques are about setting up a **port** on system which is **forwarding** traffic to a different system's socket address
- port on local system → *listening port* (LP)
- port on remote system → destination port (DP)
- with ssh, traffic is tunneled through a SSH connection



#1 - communicating to an internal service of a remote host

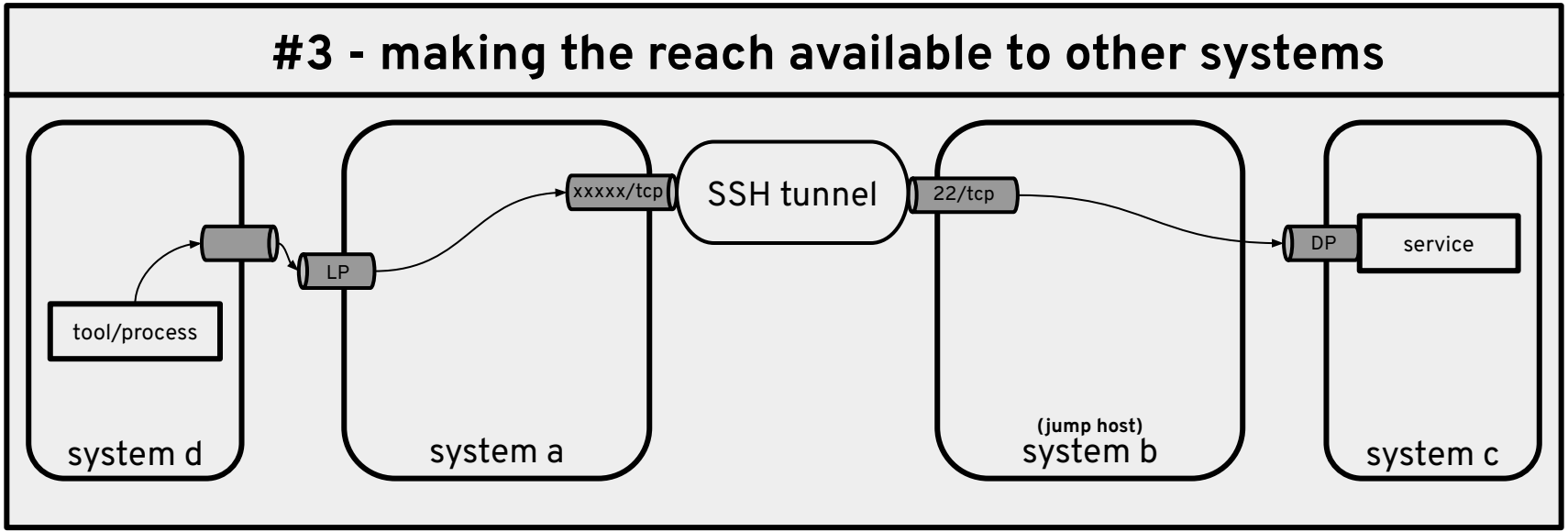


2 SSH & port forwarding in general - use case 2



"port forwarding allows to extend the reach of one system, by borrowing the reach of the system it has an SSH tunnel with."

2 SSH & port forwarding in general - use case 3



2 SSH & port forwarding in general - ProxyJump (-J)

- reaching a deep host often means hopping through a jump host that can route to it
- ssh -J (ProxyJump) chains through one or more jump hosts in a single command
  - you land directly on the target
- the SSH client stays on your box: keys/config never touch the jump host, and you authenticate end-to-end to the final target
  - note: -J forwards only the SSH login, not arbitrary tooling

```
# single jump host: hop through jumphost, land on target
[user@kali ~]$ ssh -J adam@jumphost eve@target

# chained: hop jh1 -> jh2, land on target (in order, left to right)
[user@kali ~]$ ssh -J adam@jh1,eve@jh2 frank@target
```

Pivoting

"The Art of finding a new computer to pwn after pwning the first one."

1	Motivation & Terminology
2	SSH & Port Forwarding in General
3	SSH Local Port Forward
4	SSH Dynamic Port Forward
5	SSH Remote Port Forward
6	tun2socks
7	SSH Pivoting with sshuttle
8	ligolo-ng
9	Conclusion

Tip: click any entry to jump to that section

3 SSH local port forward - intro

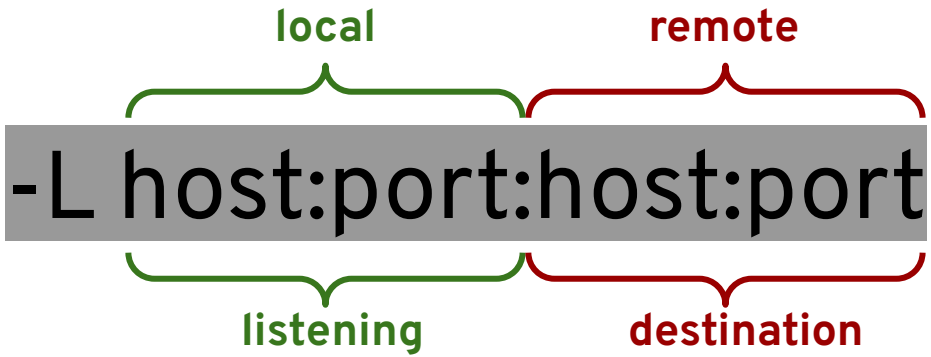
- local system has the listening port and traffic will be sent to the remote host
  - listening socket = local socket & defined from point-of-view of local system, i.e., with SSH client
  - destination socket = remote socket & defined from the point-of-view of the remote system, i.e., with SSH server

3 SSH local port forward - syntax

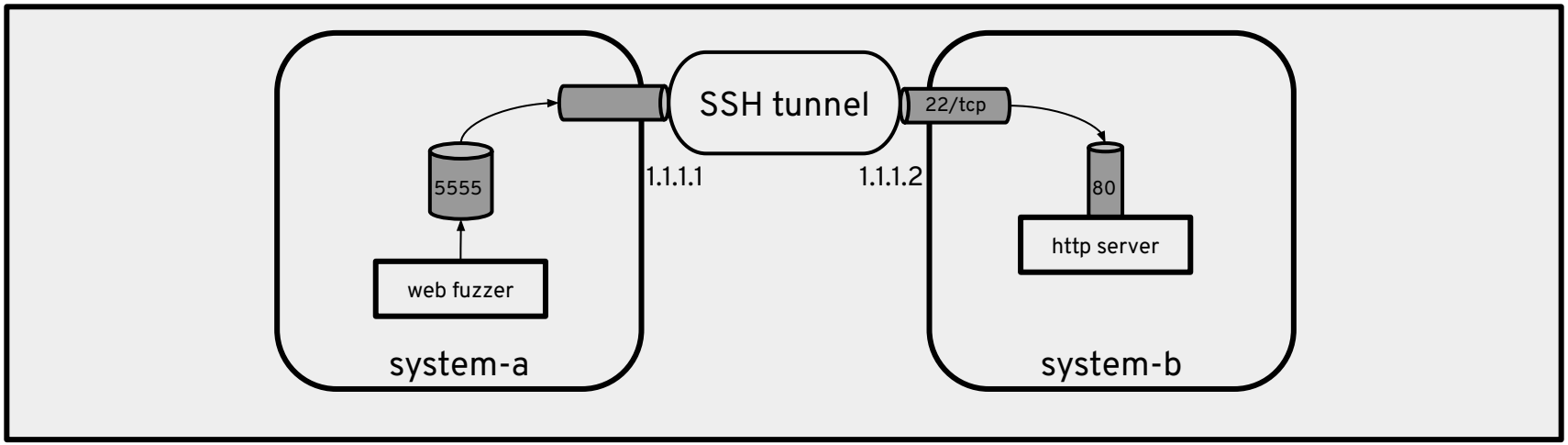
```
ssh -fNq -L local_socket:remote_socket [user@]hostname
```

- `-fNq` → put it into background (`-f`), do not open a SSH session (`-N`), do not produce any output (`-q`)
- `-L local_socket:remote_socket`
  - → `-L`: indicates a local port forward
  - → `local_socket`
    - → socket responsible for forwarding traffic (syntax: `[host:]port`)
      - → if `host` is not supplied, default value is `localhost`
    - → defined from the perspective of the local host
  - → `remote_socket`
    - → socket receiving the forwarded traffic (syntax: `host:port`)
    - → defined from the perspective of the remote host

3 SSH local port forward - syntax perspective



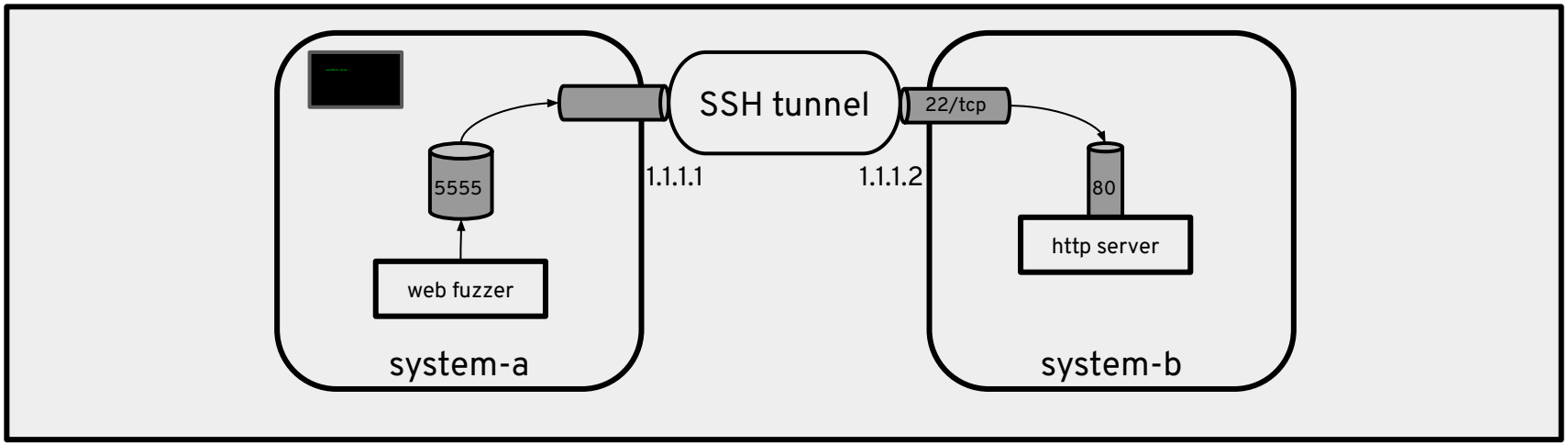
3 SSH local port forward - application for use case 1



required information

- goal is to use a local port forward to allow the web fuzzer on system-a via a listening socket `localhost:5555` to reach the http server on socket `localhost:80` of system-b
- you have credentials of `adam` on system-b

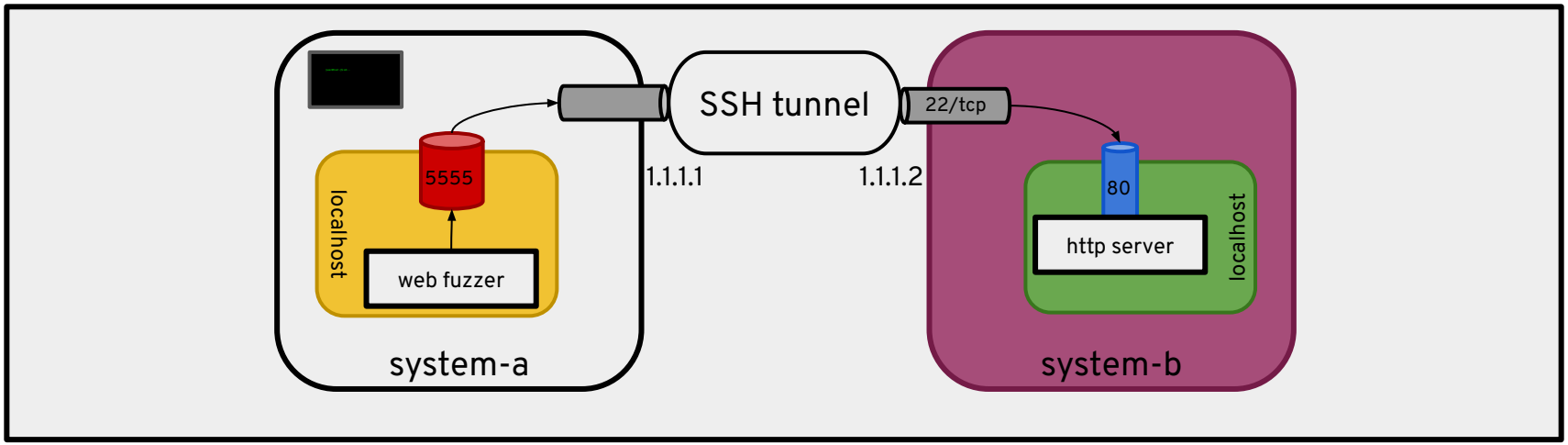
3 SSH local port forward - application for use case 1



Which of these commands will have the desired effect.

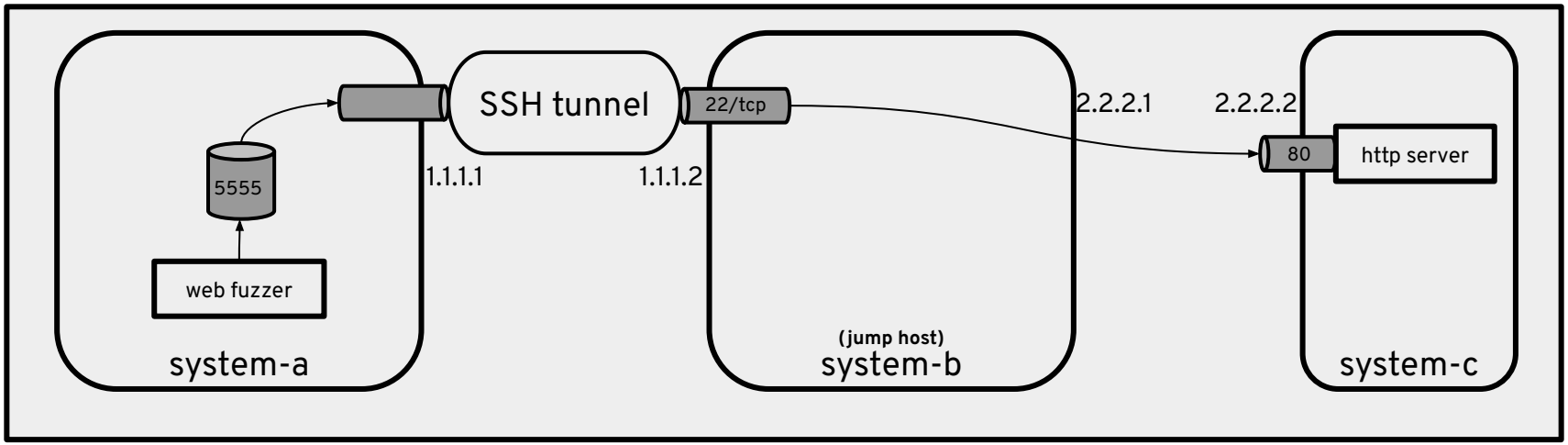
A	ssh -L 5555:localhost:80 adam@1.1.1.2	ssh -L 5555:1.1.1.2:80 adam@localhost	B
C	ssh -L 5555:1.1.1.2:80 adam@1.1.1.2	ssh -L 8080:localhost:80 adam@1.1.1.2	D

3 SSH local port forward - application for use case 1



solution: ssh -L [localhost:]5555:localhost:80 adam@1.1.1.2

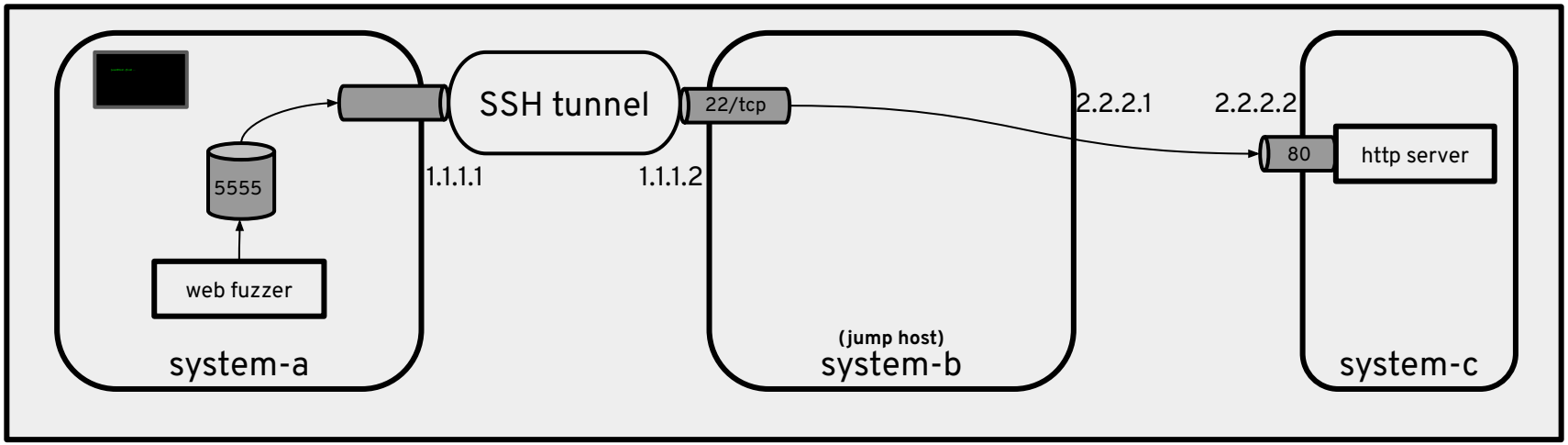
3 SSH local port forward - application to use case 2



required information

- goal is to use a local port forward to allow the web fuzzer on system-a via a listening socket `localhost:5555` to reach the http server on socket `2.2.2.2:80` of system-c
- you have credentials of `adam` on system-b and `eve` on system-c

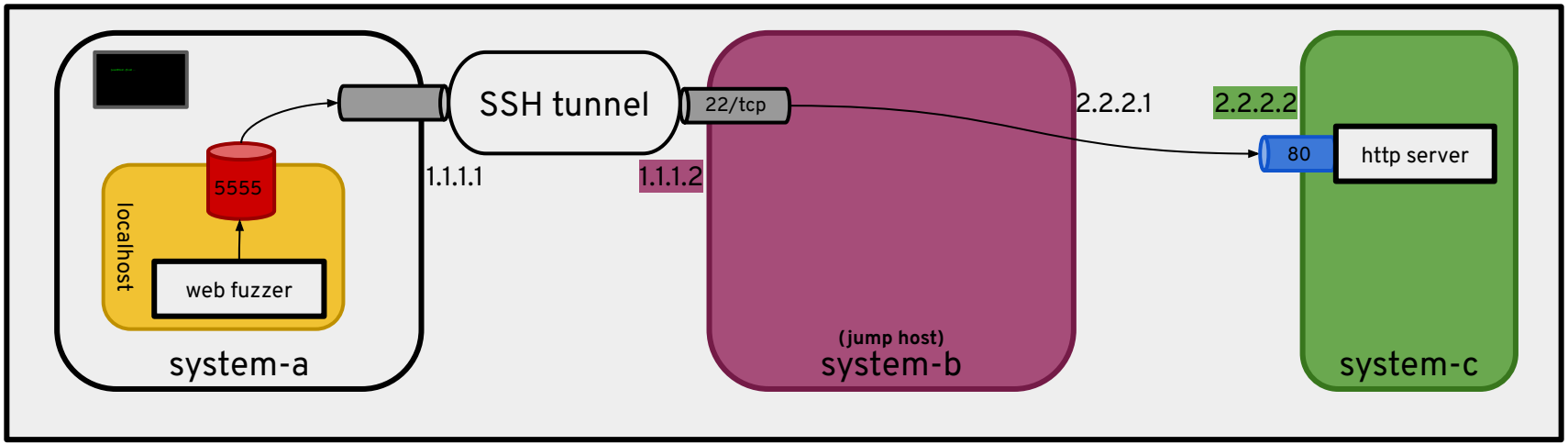
3 SSH local port forward - application to use case 2



Which of these commands will have the desired effect.

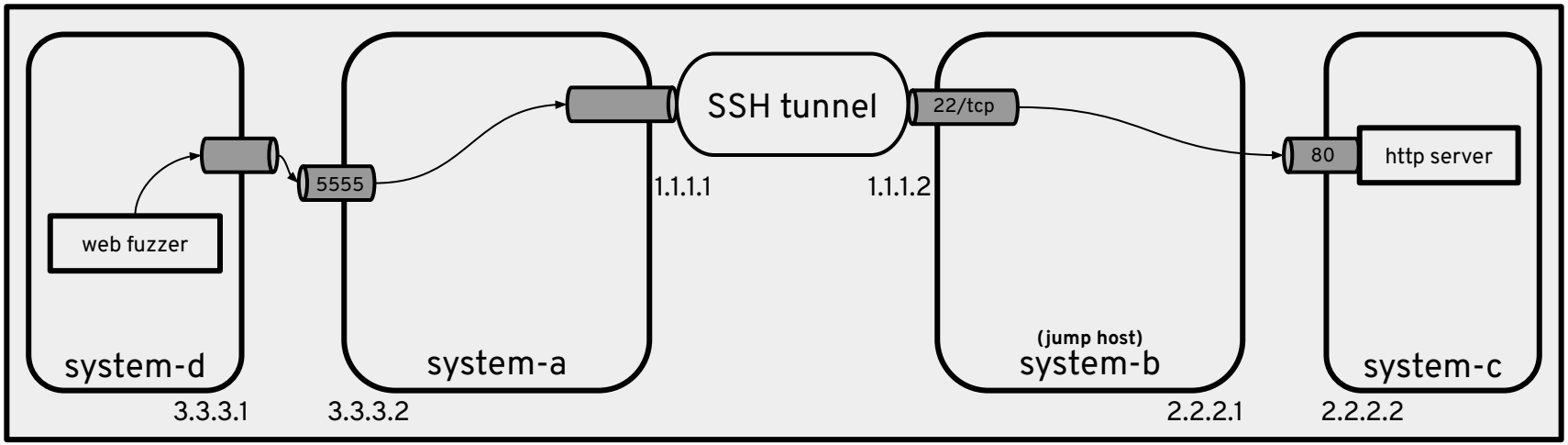
A	ssh -L 5555:2.2.2.2:80 eve@2.2.2.2	ssh -L 5555:2.2.2.1:80 adam@1.1.1.2	B
C	ssh -L 5555:localhost:80 eve@2.2.2.2	ssh -L 5555:2.2.2.2:80 adam@1.1.1.2	D

3 SSH local port forward - application to use case 2



solution: ssh -L [localhost:]5555:2.2.2.2:80 adam@1.1.1.2

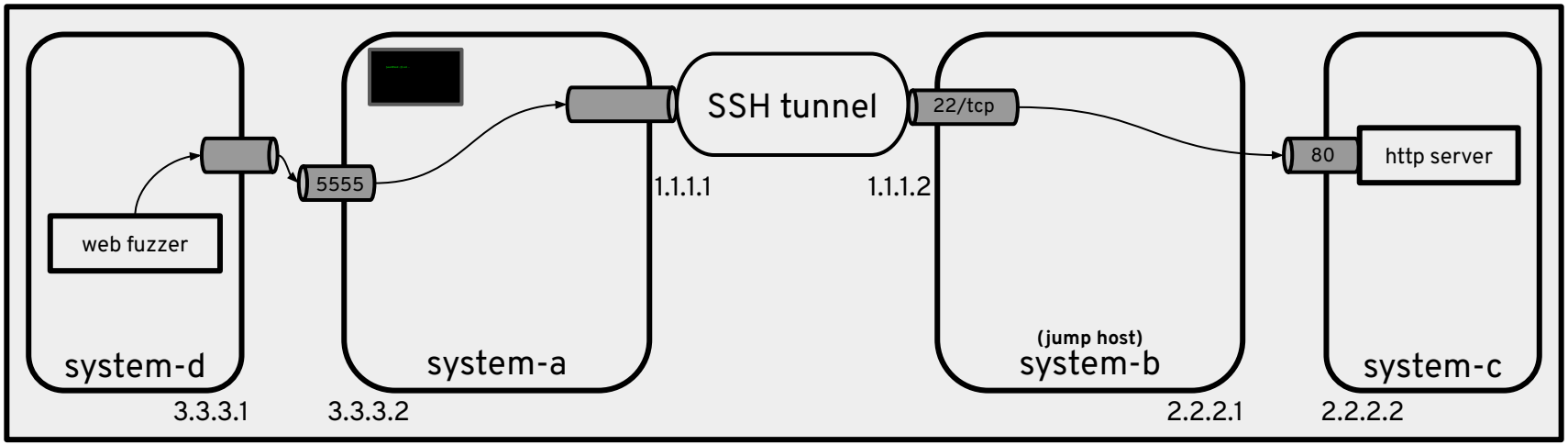
3 SSH local port forward - application to use case 3



required information

- goal is to use a local port forward to allow the web fuzzer on system-d via a listening socket `3.3.3.2:5555` to reach the http server on socket `2.2.2.2:80` of system-c
- you have credentials of `eve` on system-a and `adam` on system-b

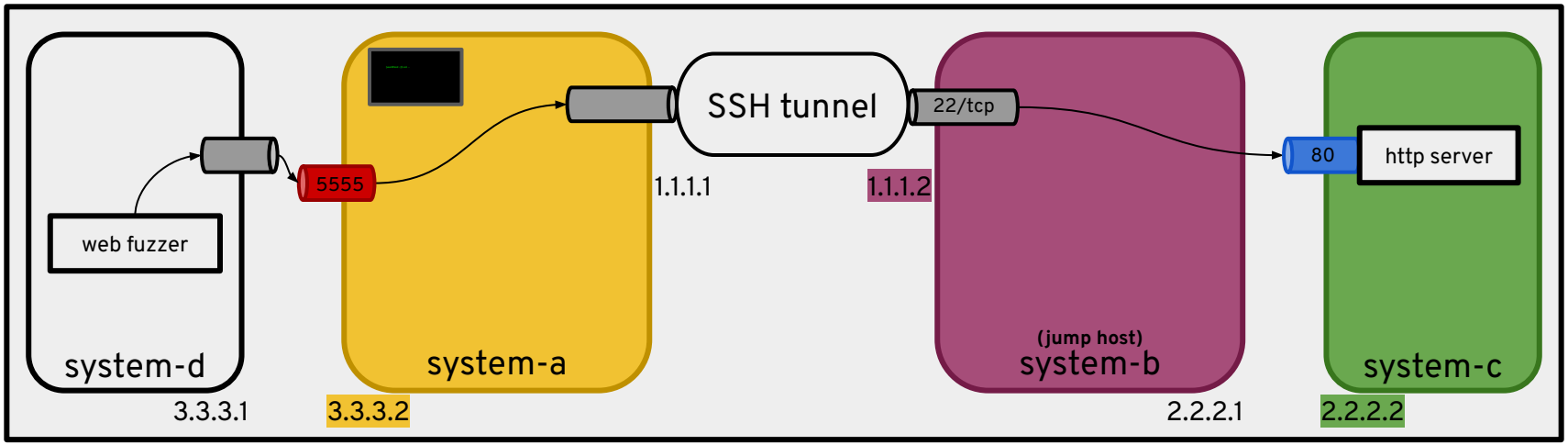
3 SSH local port forward - application to use case 3



Which of these commands will have the desired effect.

A	ssh -L 2.2.2.2:80:3.3.3.2:5555 adam@1.1.1.2	ssh -L 5555:2.2.2.2:80 adam@1.1.1.2	B
C	ssh -L 3.3.3.2:5555:2.2.2.2:80 adam@1.1.1.2	ssh -L 5555:2.2.2.2:80 eve@2.2.2.2	D

3 SSH local port forward - application to use case 3



solution: ssh -L 3.3.3.2:5555:2.2.2.2:80 adam@1.1.1.2

3 SSH local port forward - multiple forwards

- the following syntax can be used, to initiate multiple local port forwards over the same SSH tunnel

```
ssh -fNq -L local_socket1:remote_socket1
-L local_socket2:remote_socket2 [user@]hostname
```

Pivoting

"The Art of finding a new computer to pwn after pwning the first one."

1	Motivation & Terminology
2	SSH & Port Forwarding in General
3	SSH Local Port Forward
4	SSH Dynamic Port Forward
5	SSH Remote Port Forward
6	tun2socks
7	SSH Pivoting with sshuttle
8	ligolo-ng
9	Conclusion

Tip: click any entry to jump to that section

4 SSH dynamic port forward - motivation

- local port forward allows to create an encrypted one-to-one connection between sockets
- but what if we want to use a tool such as Nmap that may require access to thousands of sockets?
  - could use local port forward → tedious
  - solution: dynamic port forward

4 SSH dynamic port forward - introduction

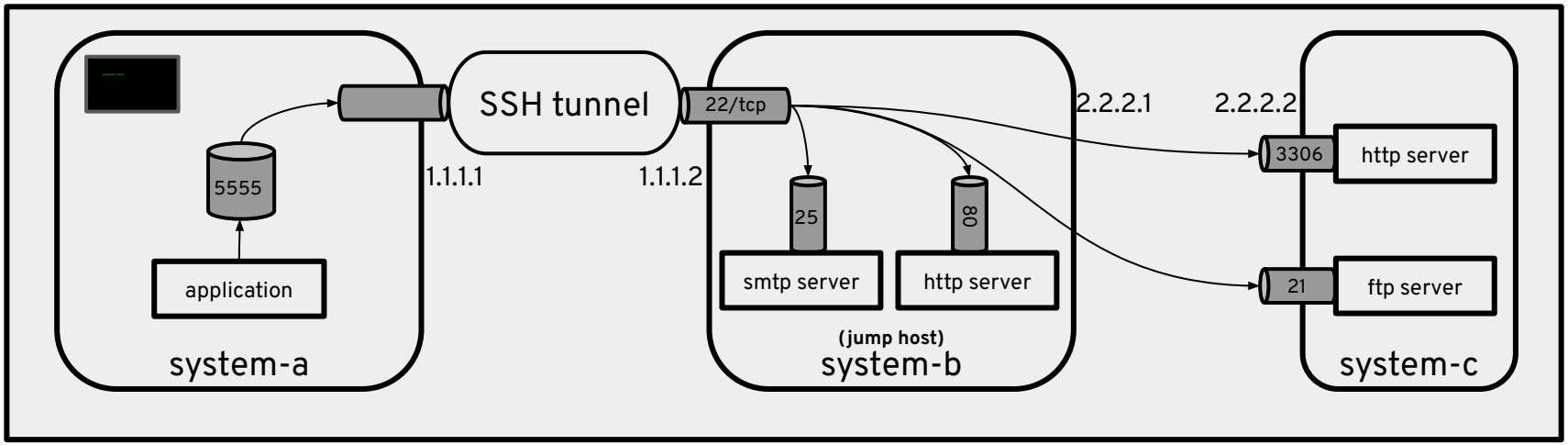
- dynamic port forward allows a one-to-many setup
  - one listening socket **on the local** host that **forwards** traffic
  - no static destination socket → **dynamic**
- listening socket hosts a **SOCKS5** proxy server
  - SOCKS (“*Socket Secure*”) channels traffic through the SSH tunnel to the appropriate destination (more on this soon)

4 SSH dynamic port forward - syntax

```
ssh -fNq -D local_socket [user@]hostname
```

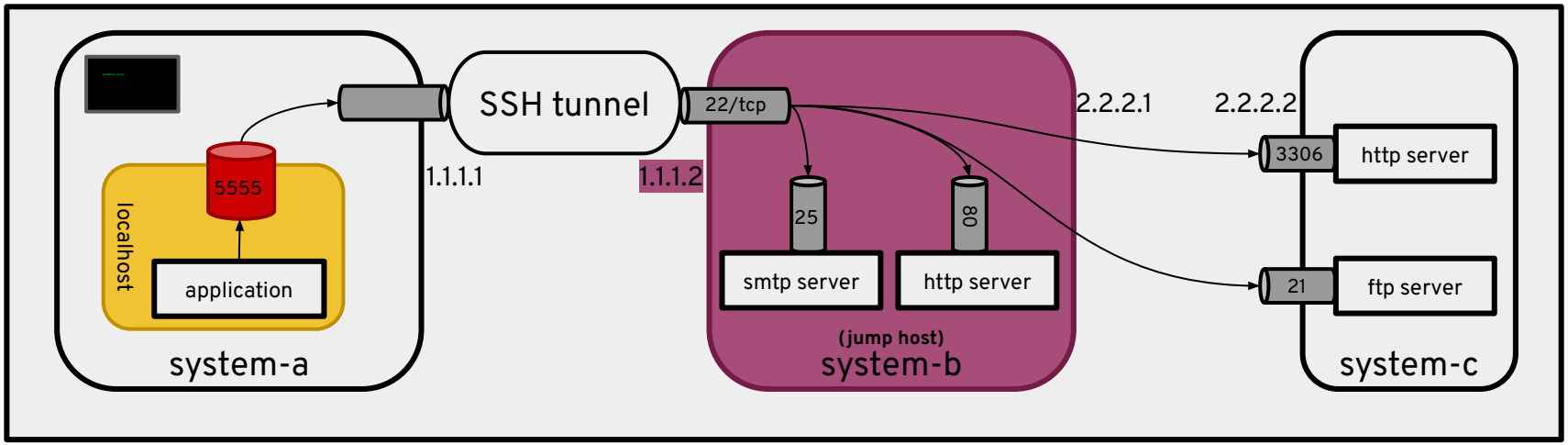
- -D local_socket
  - → -D: indicates a dynamic port forward
  - → local_socket
    - → socket responsible for forwarding traffic (syntax: [host:]port)
      - → if host is not supplied, default value is localhost
    - → defined from the perspective of the local host
    - → this socket will be occupied by the SOCKS5 proxy
  - → no (static/fixed) remote socket needs to be defined
    - → destination can be dynamically chosen

4 SSH dynamic port forward - showcase



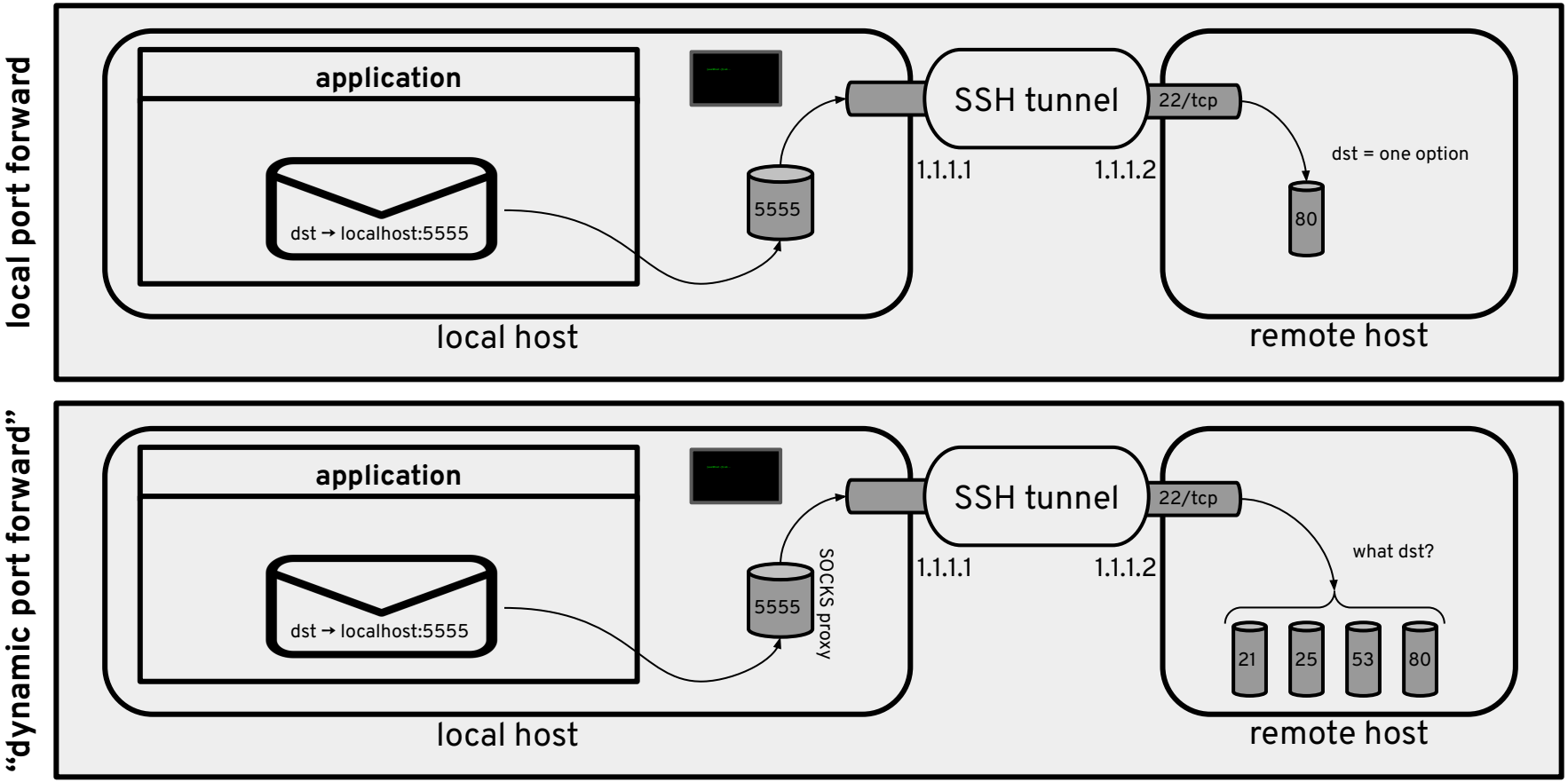

```
ssh -D 5555 user@1.1.1.2
```

4 SSH dynamic port forward - showcase

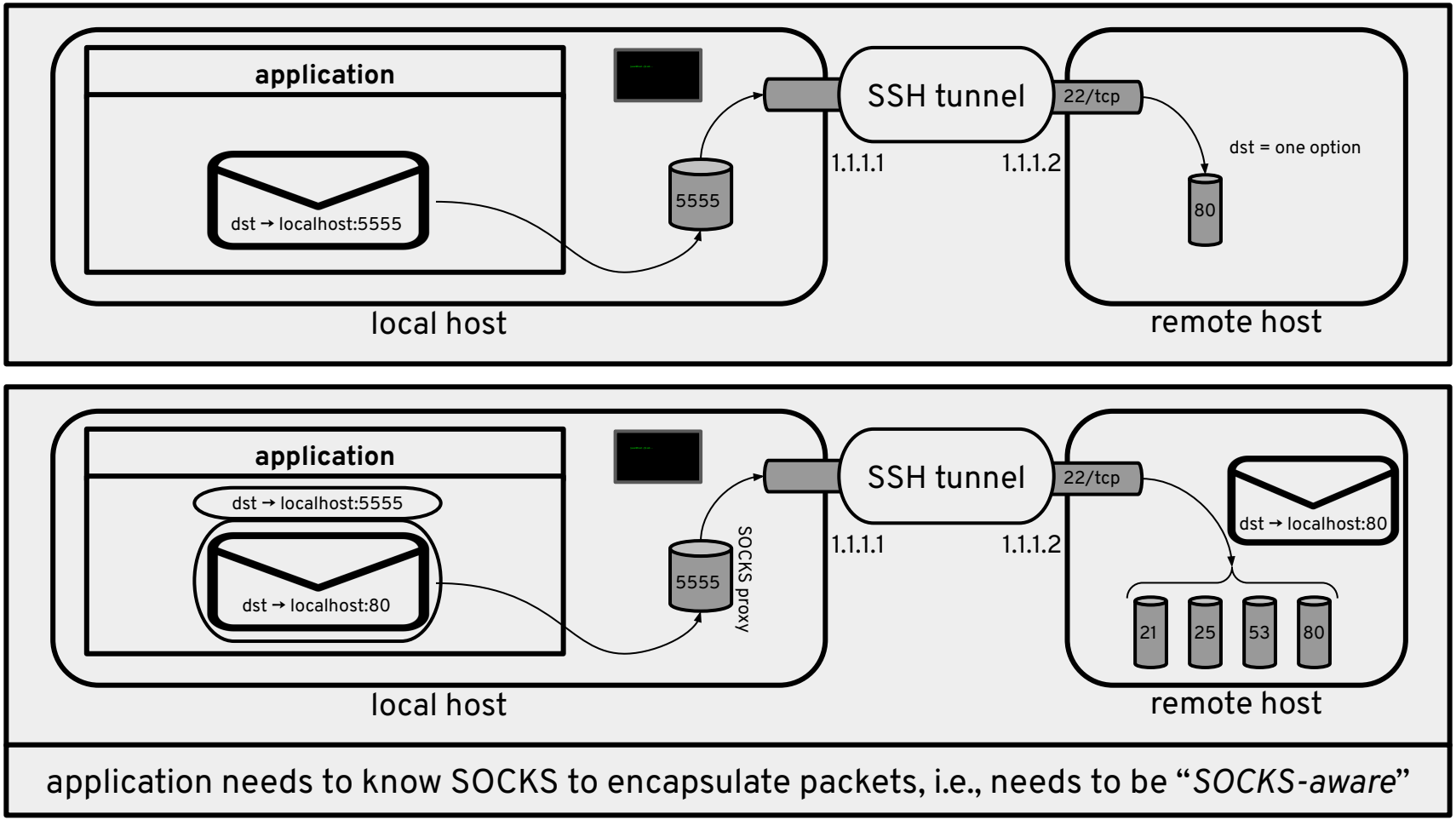



```
ssh -D [localhost:]5555 user@1.1.1.2
```

4 SSH dynamic port forward - SOCKS-aware applications



4 SSH dynamic port forward - SOCKS-aware applications



application needs to know SOCKS to encapsulate packets, i.e., needs to be "*SOCKS-aware*"

4 SSH dynamic port forward - using SOCKS5

- some utilities are by nature SOCKS-aware
  - browsers (check proxy settings),
  - `curl` (`--socks5 <proxy>:<port>`)
  - `git` (`git config --global http.proxy 'socks5://<proxy>:<port>'`)
  - ...
- most do not, which is why we need to use a tool like `proxychains`
  - forces any TCP connection by a specific application to pass through proxy server, such as SOCKS4, SOCKS5 & HTTP(S) proxies

4 SSH dynamic port forward - introducing proxychains-ng

- we will be using proxychains-ng (`proxychains` next generation)
- by default, already installed on Kali
- referenced by the command line utility `proxychains4`

4 SSH dynamic port forward - configuring proxychains-ng

```
[user@kali ~]$ tail /etc/proxychains4.conf
# add proxy here ...
# meanwhile
# defaults set to "tor"
socks4     127.0.0.1 9050

[user@kali ~]$
```

- in `proxychains4` configuration file `/etc/proxychains4.conf`, we will define the SOCKS proxy it should use, defined at the last line of the file
  - change protocol from `socks4` to `socks5`
  - change the port to listening port
- instructions online will also recommend switching from `static_chain` to `dynamic_chain` → practically no difference with a single proxy

4 SSH dynamic port forward - using proxychains-ng

```
[user@kali ~]$ proxychains4 -q command
```

- once properly configured, you do not need to address your listening socket anymore, but you “pretend” like you are in the remote network
- to use, prepend `proxychains4` in front of the command you would like to force through the SOCKS proxy of your dynamic port forward
  - `-q`: `proxychains4` will usually print a lot of output to the terminal, which we can avoid by using the “quiet” switch

- conclusion: a dynamic port forward in combination with proxychains-ng can make our life much easier, but
  - can only force TCP packets (not UDP, not ICMP)
  - not very stable and slower
    - proxy latency, network overhead

4 SSH dynamic port forward - using proxychains-ng with Nmap

```
[user@kali ~]$ proxychains4 -q nmap -Pn -sT target
```

- one of the most important use cases for dynamic port forwarding will be port scanning with Nmap
- interaction between Nmap and proxychains-ng can give rise to some “quirky” artifacts
  - scans can be (on average) 2x to 10x slower
    - solution: adapt timeout values in proxychains-ng configuration
  - host discovery is based (largely) on ICMP packets, which proxychains can not force through the proxy
    - solution: disable host discovery via the `-Pn` switch
  - proxychains-ng works by intercepting `connect` system calls of the process, which Nmap only uses when it does a full TCP connection
    - solution: always use `-sT`, other scan types do not work
- **UPDATE: proxychains and Nmap seem to be incompatible, use tun2socks instead**

Pivoting

"The Art of finding a new computer to pwn after pwning the first one."

1	Motivation & Terminology
2	SSH & Port Forwarding in General
3	SSH Local Port Forward
4	SSH Dynamic Port Forward
5	SSH Remote Port Forward
6	tun2socks
7	SSH Pivoting with sshuttle
8	ligolo-ng
9	Conclusion

Tip: click any entry to jump to that section

5 SSH remote port forward - motivation

- local port forward: reach sockets which would otherwise not be accessible
- dynamic port forward: reach multiple sockets at once
- problems that we might encounter
  - remote host does not have a SSH server running
  - firewall rules might forbid external systems to open a SSH connection to internal hosts
    - from PoV of a penetration tester: even if allowed, IDSs/IPSs/SIEMs might still be suspicious about incoming SSH connections

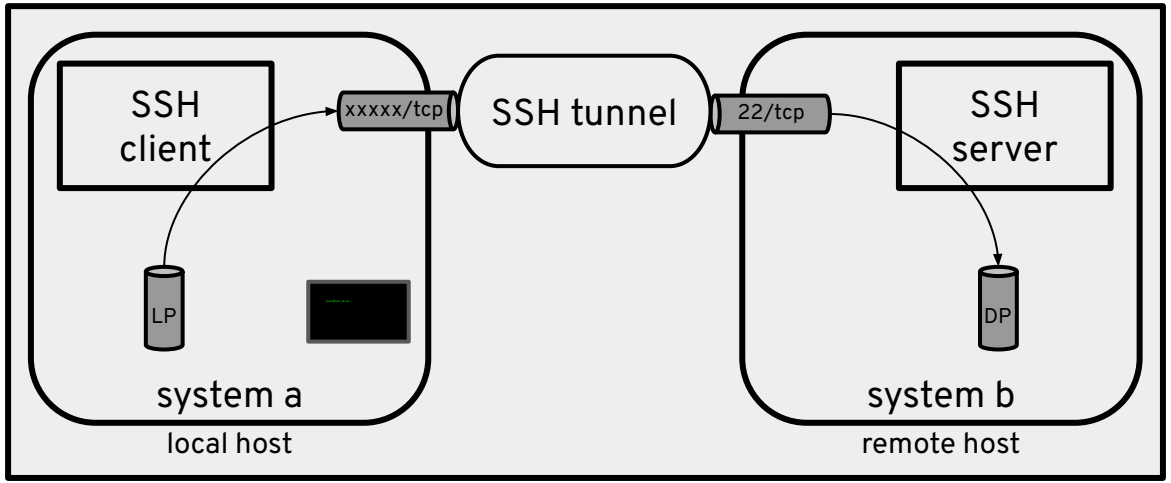
5 SSH remote port forward - intro

- a remote port forward can be the solution for those problems
  - functionally, it can do what a local port forward does (in a sense)

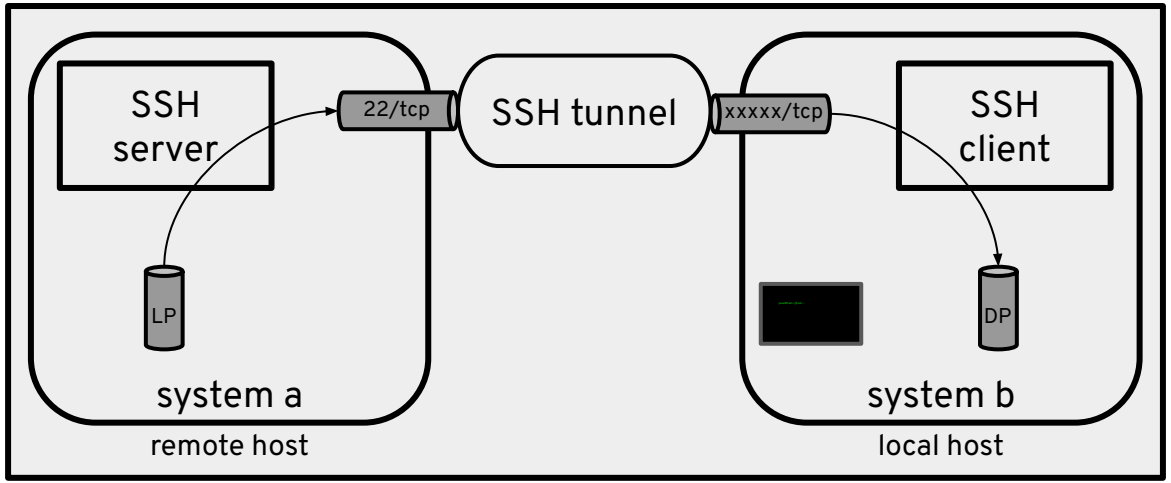
- local and dynamic port forward:
  - a local socket is listening, a remote socket is the destination
  - connection is initiated by attacker to the jumphost/target
- remote port forward:
  - a remote socket is listening, a local socket is the destination
  - connection is initiated by the jumphost/target to the attacker

5 SSH remote port forward - local vs. remote port forward

local port forward



remote port forward



"local" vs. "remote" host in the context of remote port forwarding

general point-of-view

system-a is our attacker host, making it the local host

system-b is some computer in the infrastructure of an organization, making it the remote host

SSH point-of-view

system-b functions as the SSH client from which the connection is established, making it the local host

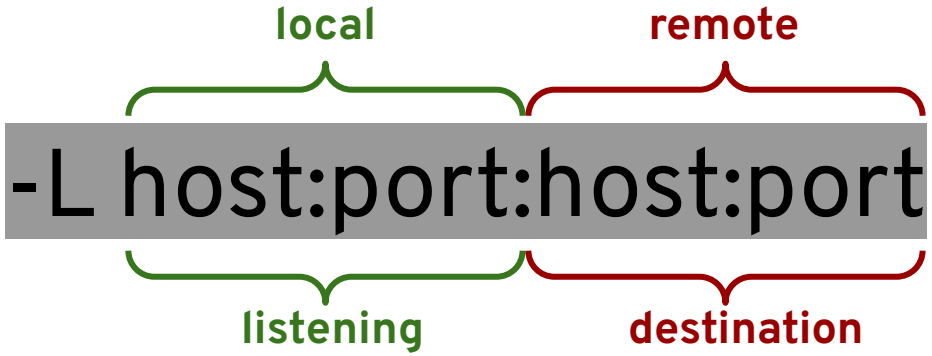
system-a functions as the SSH server which is receiving the connection request, making it the remote host

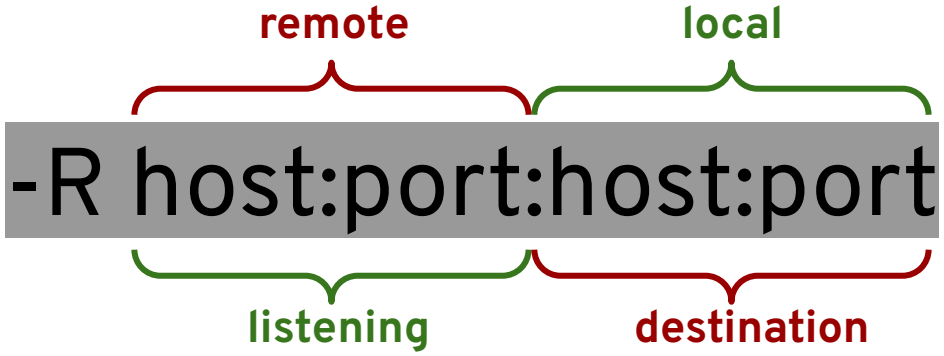
5 SSH remote port forward - syntax

```
ssh -fNq -R remote_socket:local_socket [user@]hostname
```

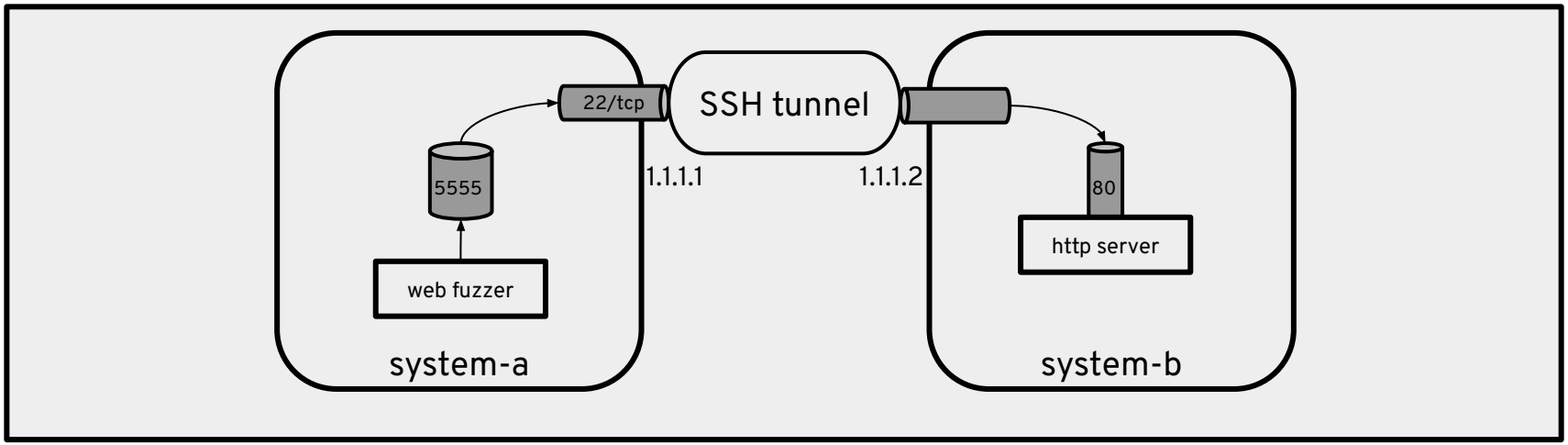
- -R remote_socket:local_socket
  - → -R: indicates a remote port forward
  - → remote_socket
    - → socket responsible for forwarding traffic (syntax: [host:]port)
      - → if host is not supplied, default value is localhost
    - → defined from the perspective of the remote host
  - → local_socket
    - → socket receiving the forwarded traffic (syntax: host:port)
    - → defined from the perspective of the local host

5 SSH remote port forward - local vs. remote port forward syntax





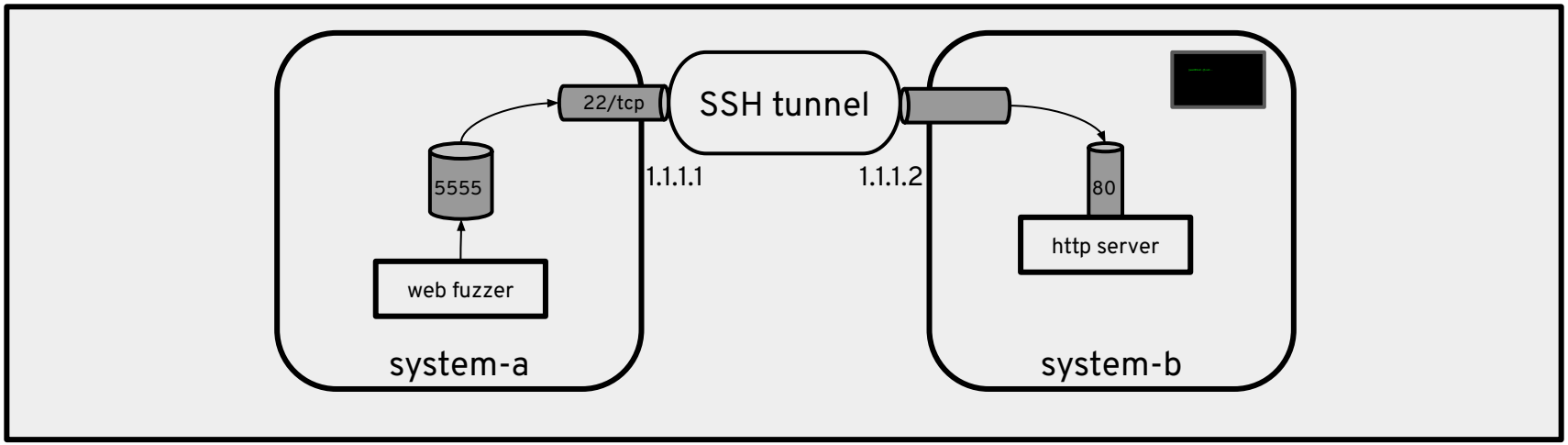
5 SSH remote port forward - application for use case 1



required information

- goal is to use a local port forward to allow the web fuzzer on system-a via a listening socket `localhost:5555` to reach the http server on socket `localhost:80` of system-b
- you have credentials of `adam` on system-a

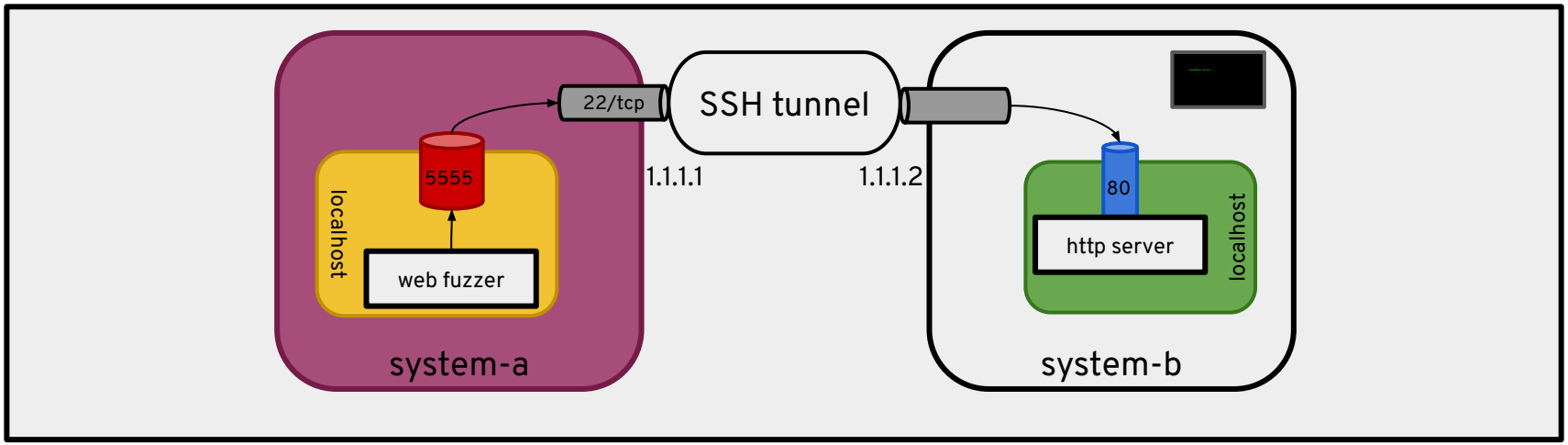
5 SSH remote port forward - application for use case 1



Which of these commands will have the desired effect.

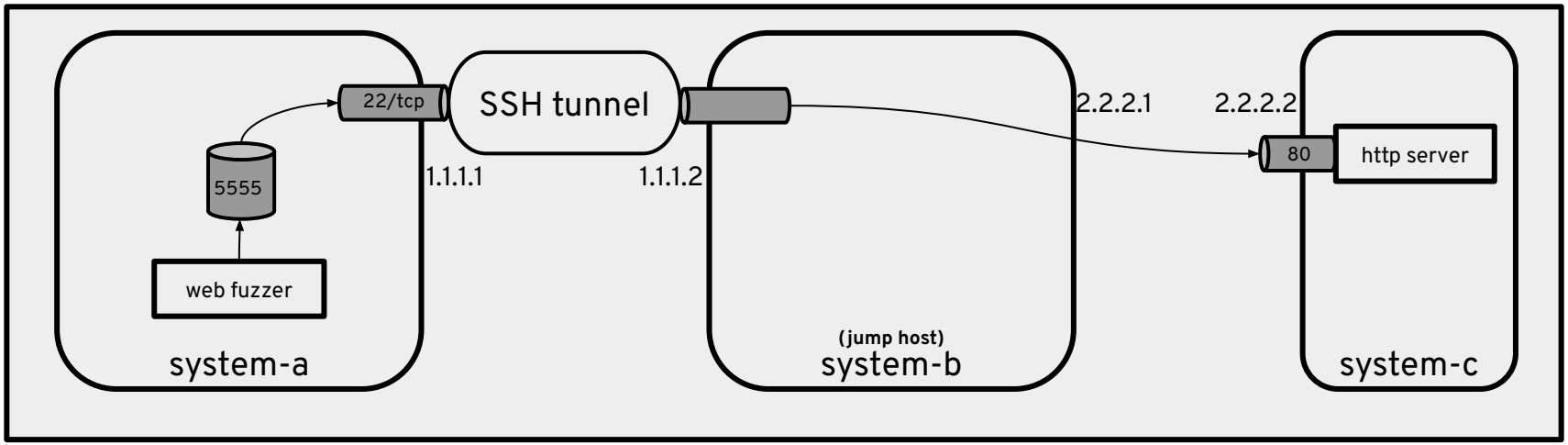
A	ssh -R 5555:1.1.1.1:80 adam@1.1.1.1	ssh -R 5555:1.1.1.1:80 adam@localhost	B
C	ssh -R 5555:localhost:80 adam@1.1.1.1	ssh -R 80:localhost:80 adam@1.1.1.1	D

5 SSH remote port forward - application for use case 1



solution: ssh -R [localhost:]5555:localhost:80 adam@1.1.1.1

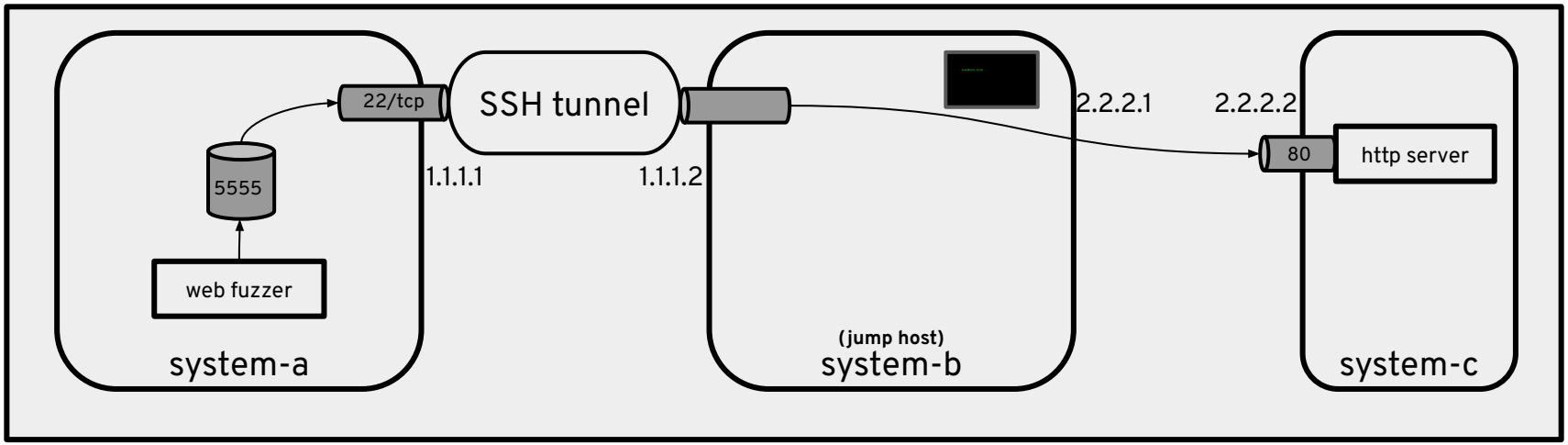
5 SSH remote port forward - application to use case 2



required information

- goal is to use a local port forward to allow the web fuzzer on system-a via a listening socket `localhost:5555` to reach the http server on socket `2.2.2.2:80` of system-c
- you have credentials of `adam` on system-a and `eve` on system-b

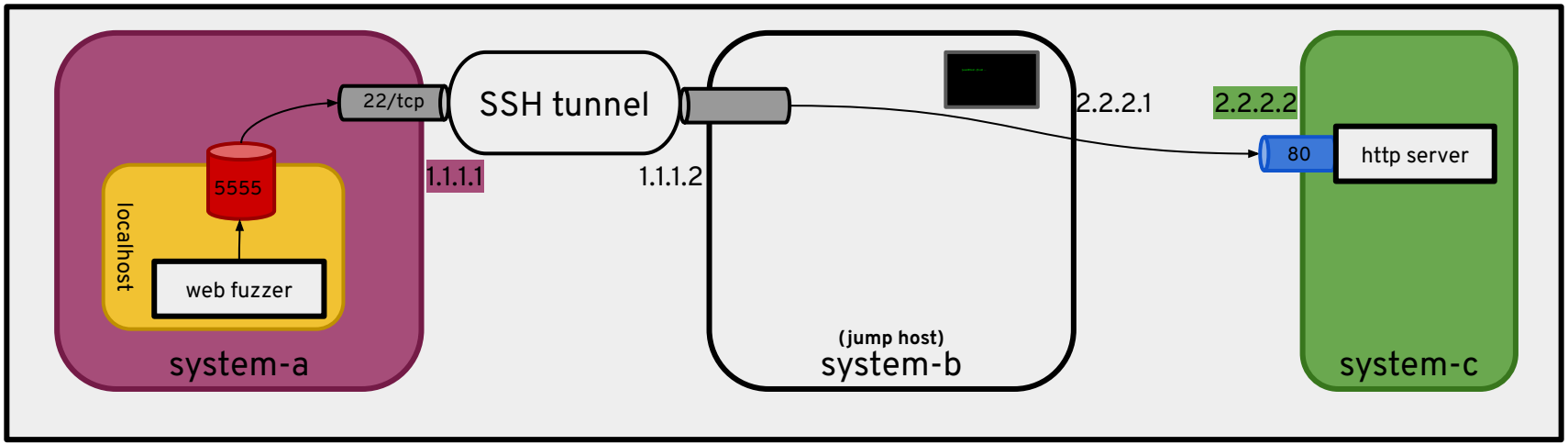
5 SSH remote port forward - application to use case 2



Which of these commands will have the desired effect.

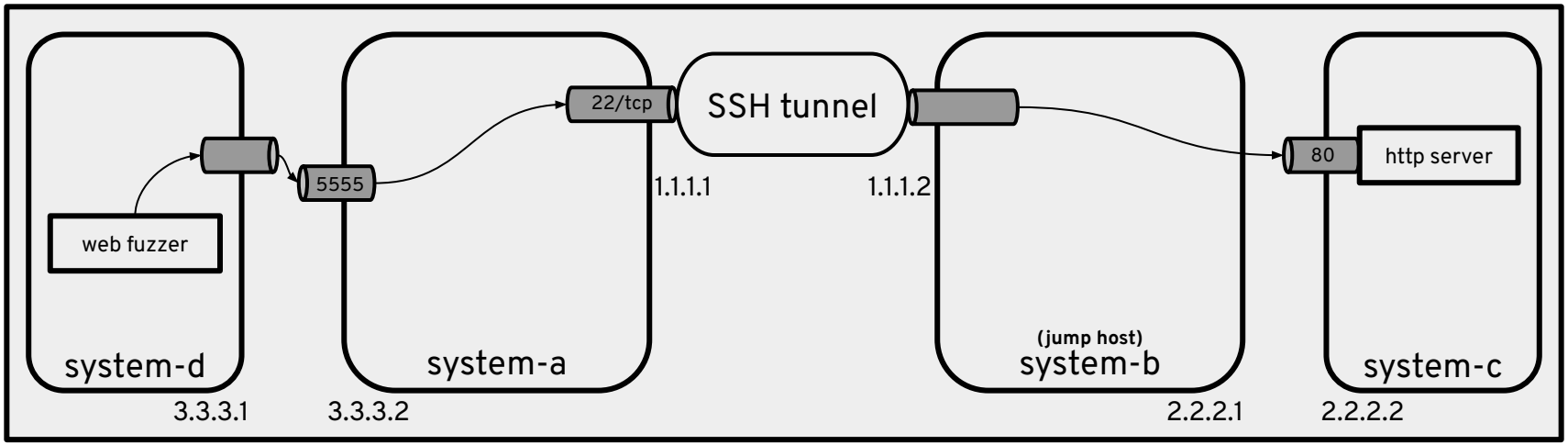
A	ssh -R 5555:2.2.2.2:80 eve@2.2.2.2	ssh -R 5555:2.2.2.1:80 adam@1.1.1.1	B
C	ssh -R 5555:localhost:80 eve@2.2.2.2	ssh -R 5555:2.2.2.2:80 adam@1.1.1.1	D

5 SSH remote port forward - application to use case 2



solution: ssh -R [localhost:]5555:2.2.2.2:80 adam@1.1.1.1

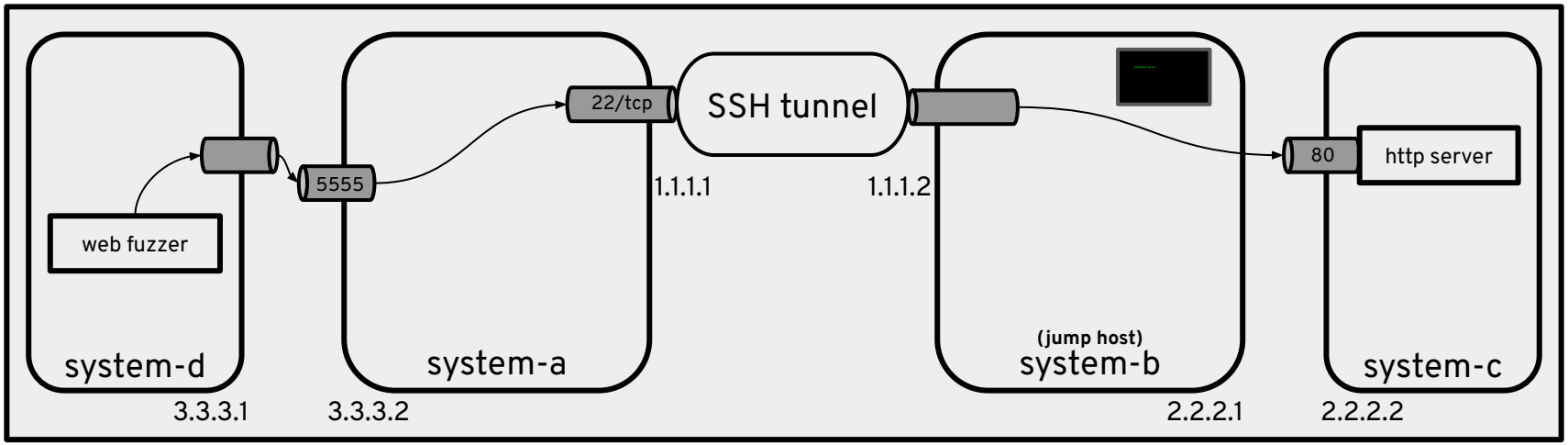
5 SSH remote port forward - application to use case 3



required information

- goal is to use a local port forward to allow the web fuzzer on system-d via a listening socket `3.3.3.2:5555` to reach the http server on socket `2.2.2.2:80` of system-c
- you have credentials of `eve` on system-b and `adam` on system-a

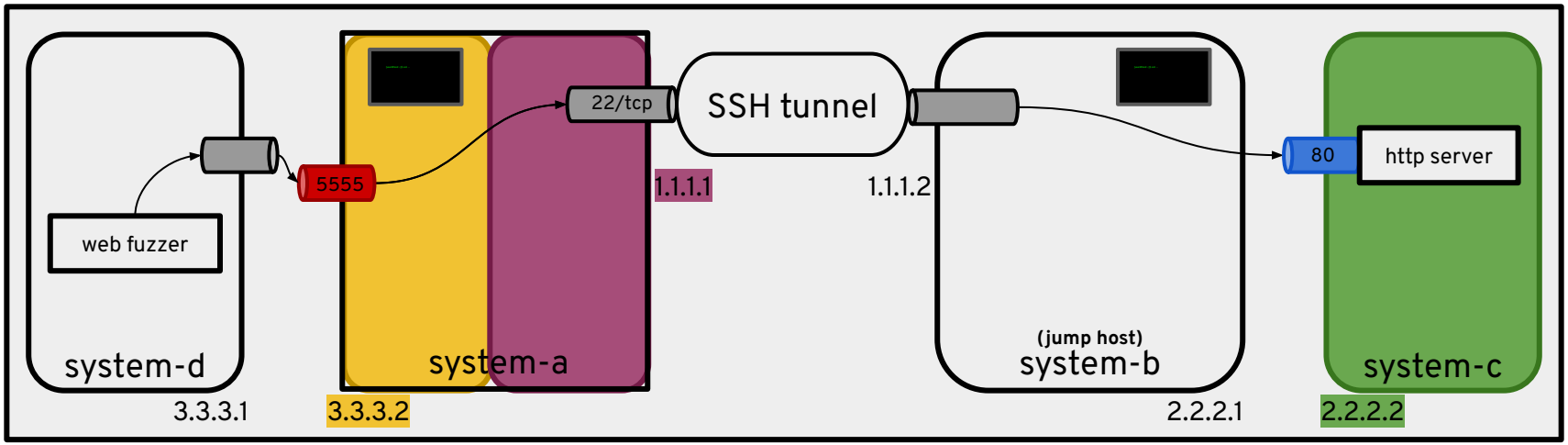
5 SSH remote port forward - application to use case 3



Which of these commands will have the desired effect.

A	ssh -R 3.3.3.2:5555:2.2.2.2:80 adam@1.1.1.1	ssh -R 2.2.2.2:80:3.3.3.2:5555 adam@1.1.1.1	B
C	ssh -R 5555:2.2.2.2:80 eve@2.2.2.2	ssh -R 5555:2.2.2.2:80 adam@1.1.1.1	D

5 SSH remote port forward - application to use case 3



solution: ssh -R 3.3.3.2:5555:2.2.2.2:80 adam@1.1.1.1

by default, this use of remote port forwarding is actually not possible

- listening socket must be attached to the `localhost` interface of the remote system
- relevant variable in the `/etc/ssh/sshd_config` of the remote host: `GatewayPorts`
  - `GatewayPorts no`: only `localhost` interface allowed (default)
  - `GatewayPorts clientspecified`: listening socket will be bound to the specified interface of the command
  - `GatewayPorts yes`: listening socket will be bound to the wildcard address `0.0.0.0` (all interfaces), regardless of what was specified in the command as the listening interface

Pivoting

"The Art of finding a new computer to pwn after pwning the first one."

1	Motivation & Terminology
2	SSH & Port Forwarding in General
3	SSH Local Port Forward
4	SSH Dynamic Port Forward
5	SSH Remote Port Forward
6	tun2socks
7	SSH Pivoting with sshuttle
8	ligolo-ng
9	Conclusion

Tip: click any entry to jump to that section

6 tun2socks - introduction

- as previously mentioned
  - dynamic port forward only works for applications that are SOCKS-aware
  - with proxychains, more applications can make use of it, but not Nmap
- one way of still making use of the dynamic proxy with Nmap is the tun2socks utility
  - basic idea: takes a SOCKS proxy and translates it to a tunnel interface

6 tun2socks - commands

- start by downloading the required tun2socks binary and using it to map the created SOCKS proxy to a (yet not created) interface

```
[user@attacker ~]$ sudo ./tun2socks -proxy socks5://127.0.0.1:5555 -device tun0
```

- tun2socks will create the required interface, but we still need to
  - activate it, i.e., set its state from *down* to *up*
  - add an IP address to the newly created interface (can be any inside of the target subnetwork)

```
[user@attacker ~]$ sudo ip link set tun0 up
[user@attacker ~]$ sudo ip addr add <ip>/<mask> dev tun0
```

6 tun2socks - interaction with Nmap

- while this setup allows Nmap to scan a target through a pivot system, there are two main issues
  - only well-formed TCP packets can be sent over interface, which means
    - avoid UDP scans (`-sU`)
    - avoid the use of scanning methods that use raw packets (e.g. `-sS`)
    - make sure to use `-Pn` flag to avoid ICMP based host-alive detection
  - regardless of whether a port is open or not, ports may appear open in the Nmap scan results
    - tunnel forwards traffic without providing the usual closed port responses (like TCP RST),
    - nmap mistakenly interprets the lack of response as the port being open
    - solution: use the `-sV` flag and check which of the open ports has an actual banner that was grabbed (very hacky...)

6 tun2socks - should you use it?

- although we did find a way of making use of the dynamic SSH port forward for Nmap, we need to realize that
  - it requires multiple steps ...
    - SSH server on jumphost
    - dynamic port forward
    - `tun2socks` binary
    - setting up interfaces/address manually
  - it only transfers TCP packets, not UDP/ICMP
  - Nmap does not work flawlessly, showing every port as open

Pivoting

"The Art of finding a new computer to pwn after pwning the first one."

1	Motivation & Terminology
2	SSH & Port Forwarding in General
3	SSH Local Port Forward
4	SSH Dynamic Port Forward
5	SSH Remote Port Forward
6	tun2socks
7	SSH Pivoting with sshuttle
8	ligolo-ng
9	Conclusion

Tip: click any entry to jump to that section

6 sshuttle - motivation

- as a possible replacement for dynamic port forward + `tun2socks`, one can also employ sshuttle
- similar to the previous approach, it uses a connection to a SSH server on a jump host to set up a pseudo VPN / interface that can be used to perform scanning with tools like Nmap (or any other type of tool)
  - while it does have the exact same problems as the previous approach (only well-formed TCP packets + all ports shown as open), it does automate the tedious set up

6 sshuttle - commands

- after downloading the required binary, the following command will set up a pseudo-VPN to the specified subnetwork

```
[user@attacker ~]$ sshuttle -r <user>@<ssh-server> <target_network>/<mask>
```

- additionally, if the target network has a DNS server, you can instruct sshuttle to also function as a DNS proxy for this server

```
[user@attacker ~]$ sshuttle --dns -r <user>@<ssh-server> <target_network>/<mask>
```

Pivoting

"The Art of finding a new computer to pwn after pwning the first one."

1	Motivation & Terminology
2	SSH & Port Forwarding in General
3	SSH Local Port Forward
4	SSH Dynamic Port Forward
5	SSH Remote Port Forward
6	tun2socks
7	SSH Pivoting with sshuttle
8	ligolo-ng
9	Conclusion

Tip: click any entry to jump to that section

8 ligolo-ng - motivation

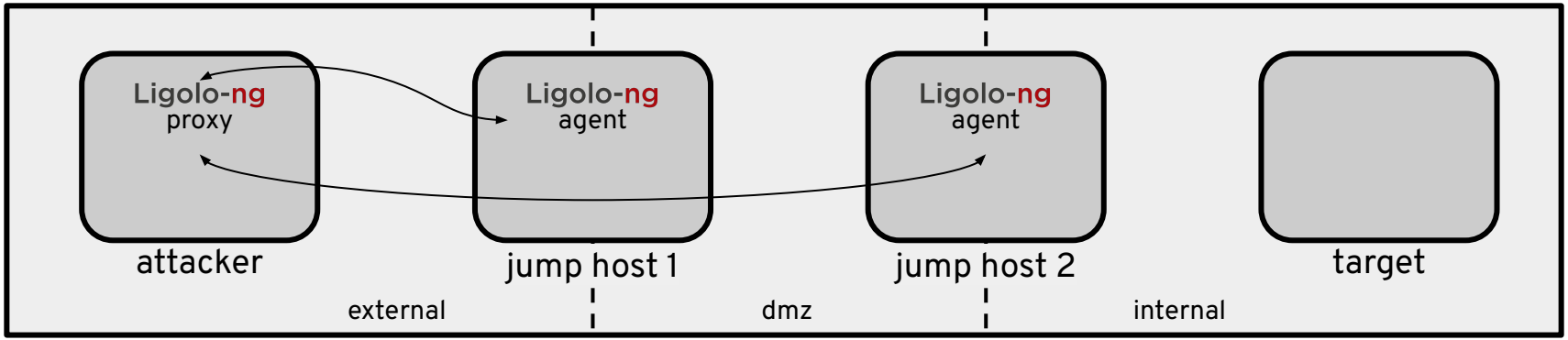
- we introduced a plethora of different pivoting techniques, each with their own drawbacks
  - **local & remote port forward**: only one destination socket
  - **dynamic port forward**: slows down traffic, tedious to set up
  - `sshuttle`: does not properly work with Nmap
  - all of them require SSH servers
- especially if we have multiple jump hosts, each of these techniques would be the wrong tool for the job
  - not flexible enough or incredibly confusing to set up
- ligolo-ng is the solution to all these problems

8 ligolo-ng - introduction

Ligolo-ng

- available here
- features
  - not based on SOCKS, instead creates a tun interface by using gVisor
  - easy to use and setup
  - encrypted communications, that are based on TLS certificates
    - automatic certificate configuration with LetsEncrypt
  - can handle multiple tunnels at once

8 ligolo-ng - methodology



- attack system has a ligolo proxy running
- each jump host will have a ligolo agent running, that will connect back to the ligolo proxy
- proxy allows to
  - configure connections
  - set up interfaces and routes
  - set up port forwards

8 ligolo-ng - general setup

- the general setup can be summarized as follows

1. download necessary binaries
2. start ligolo proxy on attacker system
3. create an interface on the attacker system
4. execute a ligolo agent on the jumphost to register to the proxy
5. add a route to the interface, associated with the target network
6. instruct ligolo, to create a tunnel to the agent system (jumphost) and to attach the created interface to this tunnel
7. (optionally) create a listener in the target network, to allow further agents to register with the proxy

8 ligolo-ng - STEP 1 - downloading necessary binaries

- proxy and agents binaries can be downloaded from the release page of the official Github



Assets 19

ligolo-ng_0.6.2_checksums.txt	1.67 KB	last month
ligolo-ng_agent_0.6.2_darwin_amd64.tar.gz	2.26 MB	last month
ligolo-ng_agent_0.6.2_darwin_arm64.tar.gz	2.16 MB	last month
ligolo-ng_agent_0.6.2_linux_amd64.tar.gz	2.19 MB	last month
ligolo-ng_agent_0.6.2_linux_arm64.tar.gz	1.99 MB	last month
ligolo-ng_agent_0.6.2_linux_armv6.tar.gz	2.04 MB	last month
ligolo-ng_agent_0.6.2_linux_armv7.tar.gz	2.04 MB	last month
ligolo-ng_agent_0.6.2_windows_amd64.zip	2.25 MB	last month
ligolo-ng_agent_0.6.2_windows_arm64.zip	2.04 MB	last month
ligolo-ng_agent_0.6.2_windows_armv6.zip	2.11 MB	last month
Source code (zip)		last month
Source code (tar.gz)		last month

Show all 19 assets

8 ligolo-ng - STEP 2 - proxy certificate

- to initiate the proxy, we simply execute the binary
- it is required, to direct the proxy on how to generate the TLS certificate it will use for encryption; two options:
  - `-autocert`: automatically requests TLS certificate from LetsEncrypt
    - requires Internet connection
  - `-selfcert`: uses a self-signed TLS certificate
    - no Internet connection required
    - will make it an untrusted source
    - allows for man-in-the-middle attacks
    - should be used only for debugging and labs

8 ligolo-ng - STEP 2 - starting the proxy

```
[user@attacker ~]$ ./proxy -selfcert
WARN[0000] Using default selfcert domain 'ligolo', beware of CTI, SOC and IoC!
WARN[0000] Using self-signed certificates
ERRO[0000] Certificate cache error: acme/autocert: certificate cache miss,
returning a new certificate
WARN[0000] TLS Certificate fingerprint for ligolo is:
8D30AC1017A1BDC0A2D98CE5A4D51B4E417ADA9434D2133F2337E86625D07777
INFO[0000] Listening on 0.0.0.0:11601
    __   _              __
   / /   (_)___ _____  / /___        ____  ____ _
  / /   / / __ `/ __ \/ / __ \______/ __ \/ __ `/
 / /___/ / /_/ / /_/ / / /_/ /_____/ / / / /_/ /
/_____/_/\__, /\____/_/\____/     /_/ /_/\__, /
        /____/                            /____/

  Made in France _          by @Nicocha30!
  Version: 0.7.5
```

- by default, the proxy will listen on all interfaces on port `11601/tcp`

8 ligolo-ng - STEP 3 - creating an interface (manually)

- on our attacker system we will have to create an interface, to which the ligolo tunnel will be attached (in fact, we will need an interface for each tunnel, as we will see later)
  - the commands below showcase the manual method; here an interface with the name `ligolo` was created

```
[user@attacker ~]$ sudo ip tuntap add user <username> mode tun ligolo

[user@attacker ~]$ sudo ip link set ligolo up

[user@attacker ~]$ ip addr show ligolo
2: ligolo: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOWN
group default qlen 500
```

8 ligolo-ng - STEP 3 - creating an interface (macro)

- to make things more efficient, ligolo comes with a set of “macros” that implement standard processes for simple usage
- these macros can be used inside the ligolo proxy prompt
  - some of these macros perform actions that require special privileges (e.g., creating an interface), so the proxy must be run with elevated privileges in these cases
- the next slide showcases all macros that are associated with managing interfaces

8 ligolo-ng - STEP 3 - creating an interface (macro)

commands: managing interfaces with ligolo macros

```
ligolo-ng » interface_create --name ligolo
INFO[0019] Creating a new "ligolo" interface...
INFO[0019] Interface created!

ligolo-ng » interface_list
┌──────────────────────────────────┐
│ Available tuntaps                │
├──────┬──────────────┬────────────┤
│ #    │ TAP NAME     │ DST ROUTES │
├──────┼──────────────┼────────────┤
│ 0    │ ligolo       │            │
└──────┴──────────────┴────────────┘

ligolo-ng » interface_delete --name ligolo
INFO[0113] Interface destroyed.
```

8 ligolo-ng - STEP 4 - starting the agent

- if we used the `-selfcert` when starting the proxy, we will have to use the `-ignore-cert` switch with the agent binary to instruct it to ignore this unsigned TLS certificate

`jumphost`

```
[user@jumphost1 ~]$ ./agent -connect attacker:11601 -ignore-cert
WARN[0000] warning, certificate validation disabled
INFO[0000] Connection established                        addr="172.26.0.2:11601"
```

`attacker`

```
ligolo-ng » INFO[0146] Agent joined.
name=user@jumphost1 remote="172.26.0.3:46596"
```

8 ligolo-ng - STEP 5 - adding and using a route

- goal is to get access to a target network, which the attacker system is not connected to but the jump host is
  - start by using the established connection to investigate the network interfaces of the jump host
- `session`: lists and allows to select ligolo sessions
- `ifconfig`: displays network information of ligolo agent system

8 ligolo-ng - STEP 5 - investigating network interfaces

```
ligolo-ng » session
? Specify a session : 1 - #1 - user@jumphost1 - 172.26.0.3:58560
[Agent : user@jumphost1] » ifconfig
...
┌─────────────────────────────────────────────┐
│ Interface 1                                 │
├──────────────┬──────────────────────────────┤
│ Name         │ eth0                         │
│ Hardware MAC │ 02:42:ac:1b:00:04            │
│ MTU          │ 1500                         │
│ Flags        │ up|broadcast|multicast|running │
│ IPv4 Address │ 172.27.0.4/16                │
└──────────────┴──────────────────────────────┘

┌─────────────────────────────────────────────┐
│ Interface 2                                 │
├──────────────┬──────────────────────────────┤
│ Name         │ eth1                         │
│ Hardware MAC │ 02:42:ac:1a:00:03            │
│ MTU          │ 1500                         │
│ Flags        │ up|broadcast|multicast|running │
│ IPv4 Address │ 172.26.0.3/16                │
└──────────────┴──────────────────────────────┘
```

8 ligolo-ng - STEP 5 - adding a route

- to instruct our system to use the created tunnel for any packets, that have their destination in the target network, we will need to create a routing rule
- below you will find both methods of doing so, the manual command and the one using a ligolo macro

```
[user@attacker ~]$ sudo ip route add 172.27.0.0/16 dev ligolo
```

```
[Agent : user@jumphost1] » route_add --name ligolo --route 172.27.0.0/16
INFO[0204] Route created.

[Agent : user@jumphost1] » interface_list
```

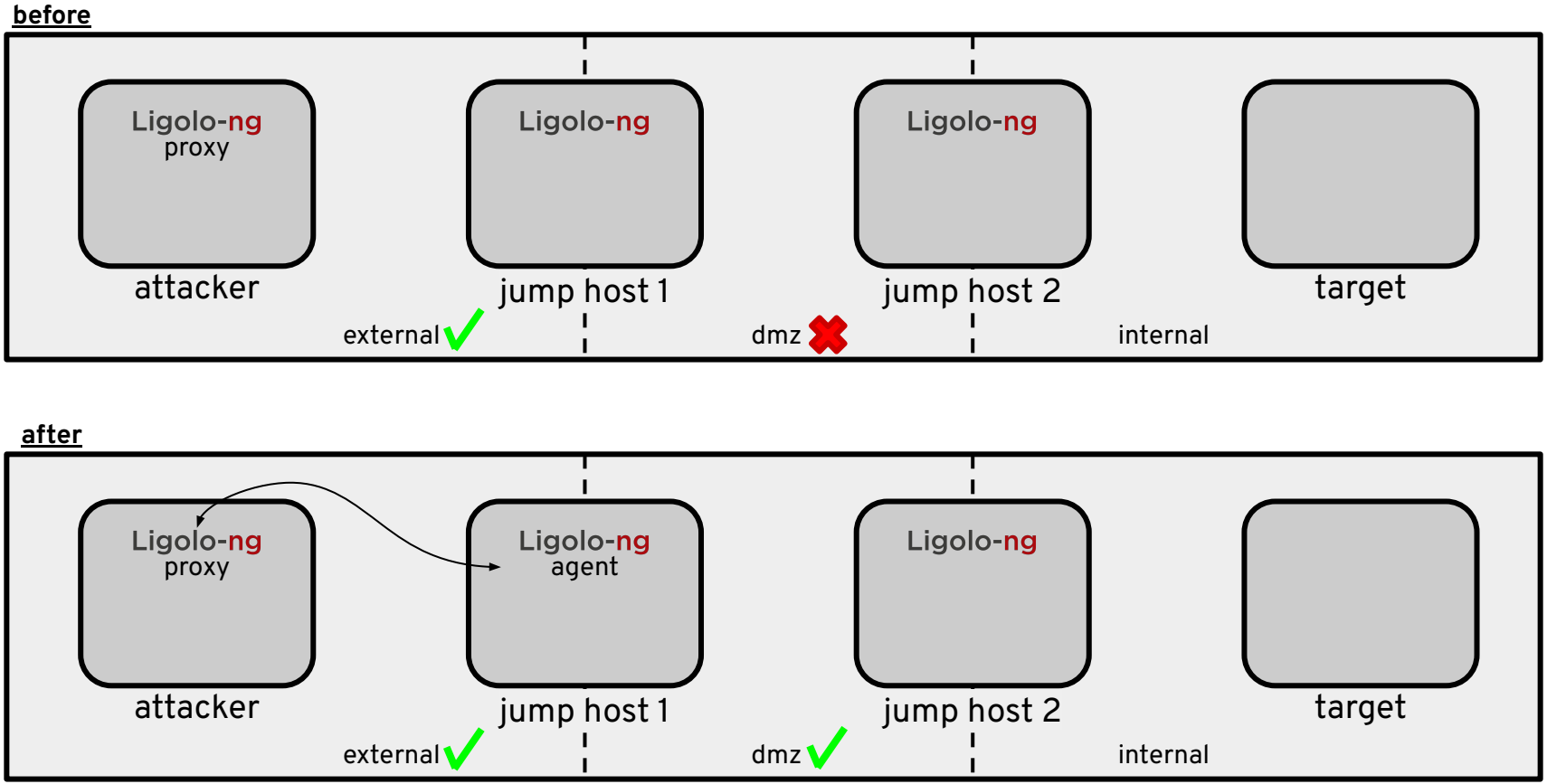
Available tuntaps		
#	TAP NAME	DST ROUTES
0	ligolo	172.27.0.0/16

8 ligolo-ng - STEP 6 - starting the tunnel

- finally, we can use the ligolo proxy to initiate the tunnel to the agent
  - while we just set the routing rule, that will instruct certain packets to use the newly created interface, we did not attach the interface with the tunnel
- both, starting the tunnel and attaching it to the interface, can be done with the following command

```
[Agent : user@jumphost1] » start --tun ligolo
INFO[0691] Starting tunnel to user@jumphost1
```

8 ligolo-ng - what is next?



8 ligolo-ng - double pivot - adding a second tunnel interface

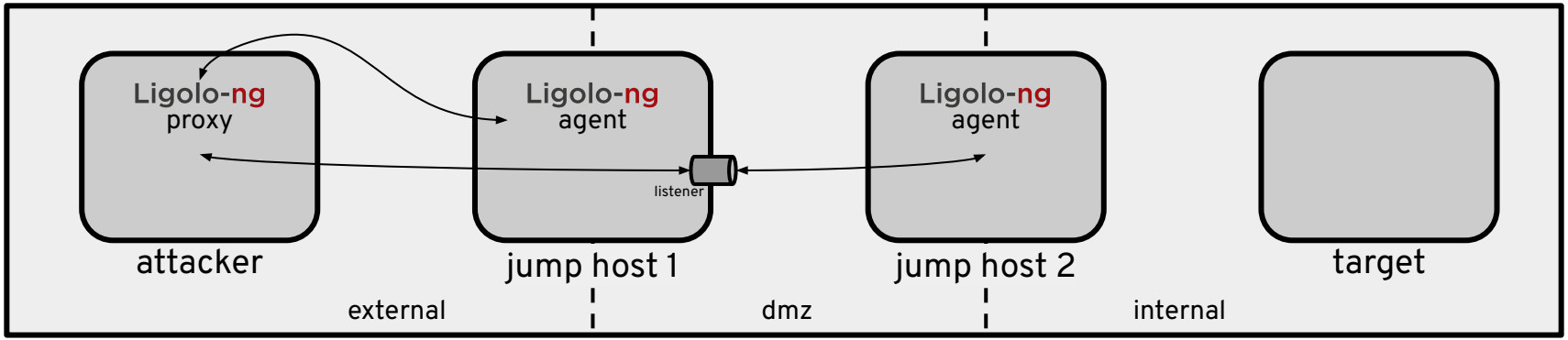
```
[user@attacker ~]$ sudo ip tuntap add user <username> mode tun ligolo2

[user@attacker ~]$ sudo ip link set ligolo2 up

[user@attacker ~]$ ip addr show ligolo2
3: ligolo2: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOWN
group default qlen 500
```

- if we want to penetrate deeper into the network, we need to pivot through an already existing pivot, i.e., a “*double pivot*”
  - requires a second ligolo tunnel
- we need to create an additional interface for this second tunnel, since every active tunnels requires its own interface

8 ligolo-ng - double pivot - necessity for a "*listener*"



- remaining challenge: the agent on jump host 2 can not contact the ligolo proxy on the attacker system
  - solution: ligolo provides functionality of a "listener"
  - listener can be set up and attached with a registered ligolo agent
  - allows for specifying one (or multiple) sockets on the ligolo agent, which will forward incoming traffic to a specified socket on the proxy
    - basically a reverse port forward (note the same as remote port forward); allows remote systems to interact with local service
  - in our case, we will instruct the agent on jump host 1 to open sockets, that forward traffic to the ligolo proxy on our attacker system

8 ligolo-ng - double pivot - adding a second tunnel interface

```
[Agent : user@jumphost1] _ listener_add --addr 0.0.0.0:11601 --to 127.0.0.1:11601 --tcp
```

- `listener_add` command allows to specify set up listener
  - listening port on ligolo agent (`--addr 0.0.0.0:11601`)
  - destination port on ligolo proxy (`--to 127.0.0.1:11601`)
- after transferring the binary to jump host 2, we are now able to connect it to the proxy as well

```
[user@jumphost2 ~]$ ./agent -connect jumphost1:11601 -ignore-cert
WARN[0000] warning, certificate validation disabled
INFO[0000] Connection established                        addr="172.27.0.4:11601"
```

- in situations, where we would like to transfer a file from our attacker system to a target which we interact with through the ligolo proxy (think privilege escalation and linpeas), we can use a listener to allow downloading from web server that is hosted on the attacker system

8 ligolo-ng - double pivot - adding a second tunnel interface

- once connected, we can add the necessary routes (just as before)
- afterwards, we select the newly added agent in the ligolo proxy prompt, allowing us to start the tunnel and attach it to the newly created interface in order to gain access to the internal network

```
[Agent : user@jumphost2] _ start --tun ligolo2
INFO[0691] Starting tunnel to user@jumphost2
```

8 ligolo-ng - how to port forward

- something, that you will not assume to be a problem until you try to make it work, is to set up a port forward that allows us to communicate with a port on the loopback interface of an agent
  - trying to communicate with a port of an agent, that is attached to an external interface is made possible, because we have access to the network that this interface is located in
    - thus, we can simply specify the address associated with this external interface inside the network
- assume, a ligolo agent hosts a webserver on `localhost:80` that you would like to access by means of ligolo
  - the only way to reference this socket is by using `localhost` or `127.0.0.1`, since it does not have any other representation
  - however, ligolo will interpret this as the loopback interface of the attacker system hosting the proxy

8 ligolo-ng - how to port forward

- solution: ligolo developer had to add a special address to reference the loopback interface of agents (documentation):
  - the subnetwork 240.0.0.0/4 was designated for that purpose, since it is officially considered unused
  - given an already registered agent with an active tunnel, the idea is that we can add a route to an IP address inside this *magic* subnet
    - for any query to an IP address on this subnetwork, ligolo will automatically redirect traffic to loopback interface of the agent, to whose tunnel we added this route

```
[user@attacker ~]$ sudo ip route add 240.0.0.1/32 dev ligolo
[user@attacker ~]$ nmap -sV 240.0.0.1
...
PORT     STATE SERVICE VERSION
139/tcp   open  netbios-ssn syn-ack Samba smbd 4.6.2
445/tcp   open  netbios-ssn syn-ack Samba smbd 4.6.2
8000/tcp open  http    SimpleHTTPServer 0.6 (Python 3.12.3)
```

8 ligolo-ng - how to port forward

- how do we allow for these loopback port forwards if we want to have them for multiple agents simultaneously?
  - simply split the magic subnetwork range into multiple disjunct networks, and attach each of these to a separate agent
    - use an IP inside of one these ranges to contact the corresponding agent's local interface

```
[user@attacker ~]$ sudo ip route add 240.0.0.1/32 dev ligolo
[user@attacker ~]$ nmap -sV 240.0.0.1
...

[user@attacker ~]$ sudo ip route add 240.0.0.2/32 dev ligolo2
[user@attacker ~]$ nmap -sV 240.0.0.2
...
```

8 ligolo-ng - using DNS server (QoL)

- we assume a situation, in which we used ligolo to set up a pivot into some internal network
  - this internal network contains a DNS/name-server, which you could use instead of pesky IP addresses
  - to enabled the use of this DNS server, add its IP to the `/etc/resolv.conf` file as you would do with any other DNS server

8 ligolo-ng - macros

- there are many different ligolo macros that might help in efficiency and ease of use
- below, you will find some of the more important

`interface_create`	`interface_list`	`interface_delete`
`ifcreate`	`iflist`	`ifdel`
`route_add`	`route_list`	`route_del`
`listener_add`	`listener_list`	`listener_del`
`tunnel_start`	`tunnel_list`	`tunnel_stop`

8 ligolo-ng - macro `autoroute`

- one new macro worth mentioning is the `autoroute` macro
- automates all necessary setup steps
- after an agent registered to the proxy, using `autoroute` in the agent session will help you in
  - create an interface
  - select and set up a route to the new network interface
  - start the tunnel and attach it to the network interface

8 ligolo-ng - macro autoroute

```
ligolo-ng » INFO[0043] Agent joined. id=42[...]34 name=user@jumphost1 remote="172.19.0.3:35864"
ligolo-ng »
ligolo-ng » session
? Specify a session : 2 - user@jumphost1 - 172.19.0.3:35864 - 42e0054b-469d-4f7f-b63e-82857424d034
[Agent : user@jumphost1] » autoroute
? Select routes to add:  [Use arrows to move, space to select, <right> to all, <left> to none, type to
filter]
> [ ]  172.20.0.2/16
  [ ]  172.19.0.3/16
```

```
[Agent : user@jumphost1] » autoroute
? Select routes to add: 172.20.0.2/16
? Create a new interface or use an existing one?  [Use arrows to move, type to filter]
> Create a new interface
  Use an existing one
```

```
[Agent : user@jumphost1] » autoroute
? Select routes to add: 172.20.0.2/16
? Create a new interface or use an existing one? Create a new interface
INFO[0274] Generating a random interface name...
INFO[0274] Creating a new "harmlesstyrannu" interface...
INFO[0274] Using interface harmlesstyrannu, creating routes...
INFO[0274] Route 172.20.0.2/16 created.
? Start the tunnel? (y/N)
```

Pivoting

"The Art of finding a new computer to pwn after pwning the first one."

1	Motivation & Terminology
2	SSH & Port Forwarding in General
3	SSH Local Port Forward
4	SSH Dynamic Port Forward
5	SSH Remote Port Forward
6	tun2socks
7	SSH Pivoting with sshuttle
8	ligolo-ng
9	Conclusion

Tip: click any entry to jump to that section

9 conclusion

local/remote SSH port forward

advantages

- straight forward
- just works
- flexibility in terms of direction (local vs remote)

disadvantages

- requires SSH server/client on jumphost
- **only “end-to-end”**

9 conclusion

~~local/remote SSH port forward~~

dynamic SSH port forward

advantages

- allows one to many

disadvantages

- requires SSH server on jumphost
- **tools need to be SOCKS-aware**

9 conclusion

~~local/remote SSH port forward~~

~~dynamic SSH port forward~~

`tun2socks`

advantages

- allows any tool to make use of dynamic port forward

disadvantages

- requires SSH server on jumphost
- only TCP; no UDP/ICMP
- **weird interaction with Nmap scans**
- **tedious setup**

9 conclusion

~~local/remote SSH port forward~~

~~dynamic SSH port forward~~

~~`tun2socks`~~

`sshuttle`

advantages

- same as dynamic SSH port forward + `tun2socks` but in one command
- allows for a DNS proxy

disadvantages

- requires SSH server on jumphost
- only TCP; no UDP/ICMP
- **weird interaction with Nmap scans**

9 conclusion

~~local/remote SSH port forward~~

~~dynamic SSH port forward~~

~~tun2socks~~

~~sshuttle~~

`ligolo-ng`

advantages

- (comparatively) straight forward and just works
- does not require any SSH server/client on jumphost
- can automate many setup steps
- TCP, UDP and ICMP
- allows for deeper pivots (*double pivot*) quite easily through functionalities such as listeners
- allows for DNS proxy (requires extra step)

disadvantages

- requires transfer of the ligolo agent binary

9 conclusion

- in conclusion, you can naturally do whatever you prefer, but my recommendation is to use…
  - local/remote port forwarding if you want to access a single port (given SSH is available)
  - `sshuttle` if you want to set up a quick and dirty pivot (given SSH is available)
  - ligolo-ng if you want to set up for proper pivoting

(Re)Sources

- https://www.ssh.com/academy/ssh/tunneling
- https://iximiuz.com/en/posts/ssh-tunnels/
- https://github.com/xjasonlyu/tun2socks
- https://github.com/sshuttle/sshuttle
- https://www.ivoidwarranties.tech/posts/pentesting-tuts/pivoting/sshuttle/
- https://github.com/nicocha30/ligolo-ng
- https://docs.ligolo.ng/sample/basic/
- https://www.hackingarticles.in/a-detailed-guide-on-ligolo-ng/